Using AgentCore Runtime in an Agentic RL Workflow

Hands-on guidance, grounded in a working slime + AgentCore project

Kaige Yang — Senior Applied Scientist, AWS

September 10, 2026

Contents

1 Introduction **2**

2 Agentic RL and its reference architecture **3**
2.1 Where agentic RL is used . . . 5
2.2 The harness — the agent loop that owns the episode . . . 6
2.3 The proxy — every model call is the policy under training . . . 8
2.4 Token-in-token-out (TITO) — the proxy's core responsibility . . . 10
2.5 slime — the trainer, and its asynchronous rollout . . . 11
2.6 Scaling the sandbox — and why inference can become the bottleneck . . . 13
2.7 Scaling the harness too — the persistent driver behind both dials . . . 16

3 Tasks frontier labs run in a sandbox **20**
3.1 Two verifier styles — verifiable and reference-based . . . 20
3.2 What the frontier reports describe . . . 20
3.3 Task families in detail . . . 22
3.3.1 Single-turn, generation-bound families . . . 23
3.3.2 Iterative, few-turn families . . . 24
3.3.3 Multi-turn, observation-heavy families . . . 24
3.3.4 Long-horizon, session-bound families . . . 25
3.4 Reading the episode shape into a config . . . 26
3.5 What every family shares . . . 26
3.6 Language and environment per family — and how one microVM serves them all . . . 27
3.7 The one family that stresses the image: SWE and the microVM's arm64 + 2 GB limits . . . 31
3.8 One image per Runtime, but a different env per row: how AgentCore reconciles it . . . 32

4 Foundations of AgentCore Runtime **34**
4.1 Runtime, session, and microVM — three concepts, precisely . . . 34
4.2 microVM vs a Docker container vs E2B . . . 35
4.3 Two compute types — managed serverless (microVM) vs Instances (GPU EC2) . . . 37
4.4 How AgentCore manages the lifecycle . . . 38
4.5 Session lifecycle in the RL loop — per-episode sessions, and why row count ≠ instance count . . . 39

4.6 Building a customized Runtime . . . 40
4.7 Example AWS repositories (use cases) . . . 42

5 Using AgentCore as the sandbox in an agentic RL workflow 42
5.1 Where AgentCore attaches to slime — hooks, not a fork . . . 42
5.2 The call path — one diagram . . . 43
5.3 Key code — three pieces . . . 43
5.4 Who controls the AgentCore lifecycle — deploy driver vs. the hook . . . 45
5.5 One harness, many task families . . . 47
5.6 The episode, end to end . . . 48
5.7 Scaling, sharing, and network isolation . . . 48
5.8 Can AgentCore host every frontier sandbox? . . . 49

6 Demos — the demo set at a glance 50
6.1 `demo_cpu_single` — MBPP code repair on a CPU microVM . . . 53
6.2 `demo_cpu_math` — math word problems on a CPU microVM . . . 54
6.3 `demo_gpu_single` — KernelBench CUDA on a GPU Runtime instance . . . 54
6.4 `demo_gpu_shared` — KernelBench, many episodes packed onto one GPU 54
6.5 `demo_cpu_quiesce` — microVM heartbeat across the weight-sync barrier 55
6.6 `demo_gpu_snapshot` — application-level snapshot / resume across a recycle . . . 55
6.7 `demo_cpu_harness_scale` — scaling the harness tier . . . 55
6.8 The per-task CPU fan-out — one recipe per real dataset . . . 56
6.9 `demo_cpu_mixtask` — one interleaved stream across task families . . . 58

7 Demos in depth — architecture, pipeline, lifecycle, and measured results 59
7.1 The shared skeleton — one pipeline, one set of metrics . . . 60
7.2 Sizing the config — response length per turn, and turns per episode . . . 63
7.3 Experiment plan — training each demo to convergence . . . 64
7.4 `demo_cpu_single` in depth — MBPP on a CPU microVM . . . 67
7.5 `demo_cpu_math` in depth — math word problems on a CPU microVM . . . 68
7.6 `demo_gpu_single` in depth — KernelBench on a GPU Runtime instance 68
7.7 `demo_gpu_shared` in depth — packing many episodes onto one GPU . . . 69
7.8 `demo_cpu_quiesce` and `demo_gpu_snapshot` in depth — surviving the pause and the recycle . . . 71
7.9 `demo_cpu_harness_scale` in depth — scaling the harness worker pool . 72

7.10 The eight per-task CPU fan-out demos in depth — one recipe per real dataset . . . 74
7.10.1 demo_cpu_gsm8k in depth — grade-school math word problems . 74
7.10.2 demo_cpu_math500 in depth — competition-style math 75
7.10.3 demo_cpu_deepscaler in depth — RL-scale math (largest split) . 75
7.10.4 demo_cpu_humaneval in depth — function-synthesis coding . . . 75
7.10.5 demo_cpu_gretelsql in depth — text-to-SQL 75
7.10.6 demo_cpu_hotpotqa in depth — multi-hop RAG 76
7.10.7 demo_cpu_xlam in depth — function-calling / tool use 76
7.10.8 demo_cpu_mcp in depth — MCP-style tool schemas 76
7.11 demo_cpu_mixtask in depth — one interleaved multi-family stream . . 77

8 Open problems in production agentic RL — infrastructure and algorithms 78
8.1 Infrastructure and systems 78
8.2 Algorithms 80
8.3 Takeaways 81

9 Summary 81

10 References — papers & models 82

11 References — AgentCore docs (1/2) 82

12 References — AgentCore docs (2/2) 82

13 References — AgentCore SDK, CLI & samples 83

14 References — production agentic RL (open problems) 83

1 Introduction

This report is a **hands-on guide for AWS customers doing agentic RL**. It is grounded in a working slime + AgentCore project and in the **official AgentCore docs** [D1].

After reading it, you will know:

- **The reference architecture of agentic RL** — how the trainer, inference engine, proxy, harness, and sandbox fit together, and which seams AgentCore fills (§“Agentic RL and its reference architecture”).
- **What tasks to train, and how the reward works** — the verifiable task families frontier labs run in a sandbox, each scored by a programmatic **RLVR** checker, not an LLM judge (§“Tasks frontier labs run in a sandbox”).
- **What AgentCore can and cannot do** as an RL sandbox (and, optionally, harness), and **how to build and customize** your own Runtime — CPU (Code Interpreter) and GPU (Instances) (§“Foundations of AgentCore Runtime”).
- **What AgentCore *is*, precisely** — the three concepts you must not conflate (**Runtime**, **session**, **microVM**), their **1 : N : 1** cardinality, the two compute types (serverless CPU microVM vs GPU **Instances**), and how the **lifecycle** works — implicit launch on first invoke, the **~15-minute idle timer**, and implicit termination on idle / max-lifetime / failed health check (§“Foundations of AgentCore Runtime”).
- **How to run a real task on it** — datasets, Dockerfiles, deploy commands, and the single slime hook that wires AgentCore in without a fork (§“Using AgentCore as the sandbox in an agentic RL workflow”). The project ships a **sixteen-demo set**, presented at two levels: a brief one-row-per-demo catalogue (§“Demos — the demo set at a glance”) and a full per-demo walkthrough of architecture, pipeline, lifecycle, and **measured results** (§“Demos in depth”).
- **How to train agentic RL on a GPU Runtime** — an AgentCore Runtime **Instance backed by GPU-based EC2** (g6 / L4), where untrusted CUDA builds and runs against a checker on a disposable GPU (`demo_gpu_single`, `demo_gpu_shared`; §“Demos in depth”).
- **How to scale the loop** — measure the per-step bottleneck, then scale the three tiers **jointly**: inference, the sandbox, **and the harness itself**, because rollout throughput is `min(inference, sandbox, harness)`; and **balance load correctly per side** — a cache-aware SGLang router for inference, sticky round-robin by session id for the sandbox pool (§“Scaling the sandbox”, §“Scaling the

harness too").

- **How to keep a sandbox live across a weight sync** — a **heartbeat ping (health check)** holds the session warm while the trainer and inference engine sync weights, resetting the idle timer so the *same* sandbox is reused with no cold start per round (`demo_cpu_quiesce`).
- **How to snapshot and resume at the application level** on a GPU-EC2 Runtime Instance — a **Dockerfile + a custom script, triggered by the weight-sync event**, checkpoints application state to a store you own, so training resumes after a recycle from the saved snapshot (`demo_gpu_snapshot`).
- **Where the open problems are** — the infrastructure and algorithmic questions production agentic RL still faces (§"Open problems in production agentic RL").

Scope. AgentCore = the execution **sandbox**; **slime** = the RL trainer; the **harness stays in-process** (warm, next to the trainer) — though the report shows the harness is itself a scaling dimension, replicated as warm workers rather than made ad-hoc. The through-line is the sandbox seam — build image → deploy Runtime → seed a task → invoke → read the reward — and then scaling the loop around it and surviving its stalls.

Roadmap. The report proceeds in order: the **reference architecture** of agentic RL and how slime drives it (§2); the **verifiable task families** and their rewards (§3); the **foundations of AgentCore Runtime** — concepts, compute types, and lifecycle (§4); **wiring AgentCore into slime** as the sandbox, with the call path and key code (§5); the **demo set at a glance** (§6) and then **in depth** with measured results (§7); the **open problems** that remain (§8); and a closing **summary** of what AgentCore can and cannot do (§9).

2 Agentic RL and its reference architecture

What is agentic RL? In *agentic* reinforcement learning the model under training is not asked for a single answer — it acts as an **agent** over many turns against an **executable environment**: it writes code, runs a command, reads the result, and tries again. Only when the trajectory ends does a **programmatic checker** turn the outcome into a scalar reward (tests pass, result-set matches, flag verified) — **no LLM judge** [1, 2]. This is reinforcement learning with *verifiable* rewards (RLVR): the environment, not a model, decides who was right.

Training that loop end-to-end uses a **disaggregated** design (following Agent Lightning [1]): a training side (trainer + inference engine) and an agent side (harness + sandbox), joined by an LLM-proxy / rollout-collection gateway. The reference architecture below traces the full cycle — a **generation path** across the top and a **training-data path** back along the bottom.

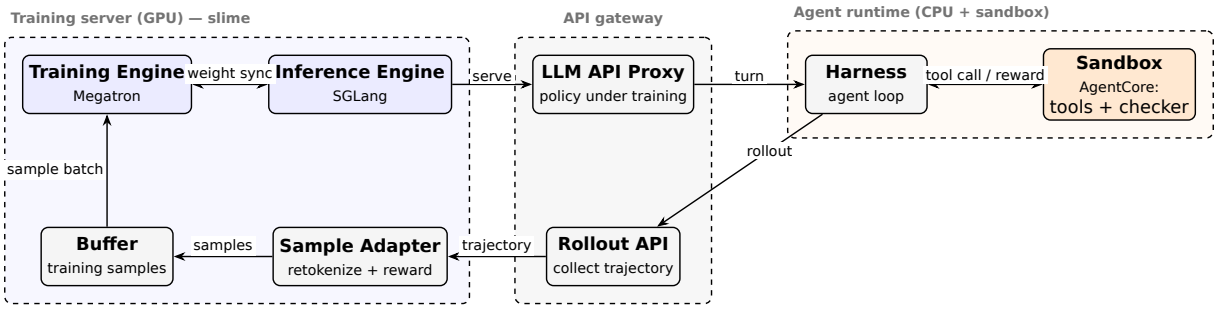


Figure 1: Reference architecture for (harnessed) agentic RL, following the disaggregated design of Agent Lightning [1] with slime [9] as the training server. Three domains: a **training server** (GPU), an **API gateway**, and an **agent runtime**. **Generation lane (top, $\rightarrow$):** the training engine syncs weights to the inference engine; the LLM API proxy fronts inference so every turn is served by the policy under training; the harness drives the agent loop; the sandbox (AgentCore) runs each tool call and the checker. **Training-data lane (bottom, $\leftarrow$):** the rollout is collected via the Rollout API, the Sample Adapter retokenizes it and attaches the reward to form training samples, which accumulate in the Buffer; the trainer draws a sample batch and updates — closing the loop.

The core components.

- **Trainer** — the learner (here **slime** driving **Megatron**, on GPU). It samples prompts, collects finished trajectories with their rewards, computes the policy-gradient update, and periodically pushes fresh weights to the inference engine.
- **Inference engine** — the generation server (here **SGLang**, on GPU) that produces the model's turns at rollout speed. It holds the *current* policy weights; the trainer refreshes them via **weight sync** so rollouts always reflect the latest policy.
- **Proxy** — a thin adapter (CPU) fronting the inference engine so that every model call the harness makes is answered by the **policy under training** (not some external API). It is the seam that lets an off-the-shelf agent loop drive an in-training model.
- **Harness** — the **agent loop** (CPU): it holds the conversation, calls the proxy

for each turn, extracts the model's proposed action (a code block, a tool call), submits it to the sandbox, and feeds the result back as the next observation until the episode ends.
- **Sandbox** — the **isolated execution environment** (CPU, or GPU for CUDA work) where the model's untrusted output actually runs (write → run → read) and where the **checker scores the reward**. It must be ad-hoc and isolated per episode.

The training-data path. The bottom of Figure 1 shows how a finished episode becomes a gradient step. The **Rollout API** collects the trajectory (the sequence of LLM request–response pairs the harness made); the **Sample Adapter** retokenizes those calls and attaches the episode's reward and advantage, turning one rollout into one or more **training samples**; those land in the **Buffer**; and the trainer draws a **sample batch** from the buffer for each update. In *harnessed* agentic RL the training engine never sees the environment directly — only this stream of samples reconstructed from the proxy's request/response log [1].

2.1 Where agentic RL is used

The same loop — act in an environment, score with a checker — covers a wide range of tasks. The common thread is a **verifiable reward**: an environment (not a human or an LLM judge) decides success. Representative use cases:

- **Software engineering (SWE)** — edit a real repository to resolve an issue; reward = the project's test suite turns green (e.g. SWE-bench [6]).
- **Code generation & competitive programming** — write a function that passes hidden unit tests (e.g. MBPP [8]).
- **GPU-kernel / systems** — write a CUDA kernel and optimize it; reward = numerical `allclose` plus a speedup target (e.g. KernelBench [7]).
- **Math & logical reasoning** — produce an answer checked for exact/numeric equality against ground truth.
- **Tool use / function calling** — orchestrate multi-step API or tool calls; reward = correct final result *and* the right tools invoked.
- **SQL & data analysis** — generate a query whose result-set matches the expected rows.
- **RAG & search agents** — retrieve then answer; reward = answer grounded in the retrieved facts.
- **Web & computer use** — drive a browser or GUI to complete a task.

- **Security / CTF** — exploit a challenge and capture the flag; reward = the checker validates the flag.
- **Multi-agent & long-horizon workflows** — several agents or many turns cooperate toward a verifiable end state.

Every one of these needs an **isolated place to execute untrusted output and run the checker** — the sandbox seam this report is about.

2.2 The harness — the agent loop that owns the episode

What it is. The harness is the **agent loop**: the orchestration code that runs *one* episode end to end. In this project it is `run_episode` (in `harness.py`), and it is **task-agnostic** — the same loop serves every task family; only the task's prompt, seed files, and checker change. We keep it **in-process** (a warm Python coroutine next to the trainer), because episodes are independent and the loop needs *warmth*, not isolation — that isolation belongs to the sandbox.

How it works with the other components. The harness sits between the proxy and the sandbox and is the only component that sees a whole episode:

- **↔ Proxy:** for each turn it sends the running conversation to the proxy and receives the model's response (the policy under training).
- **↔ Sandbox:** it extracts the model's proposed action (a code block, a tool call) and executes it in the sandbox, then reads back stdout / exit-code as the next observation.
- **↔ Task / checker:** it seeds the workspace from the task and, at the end, runs the checker (in the sandbox) to get the reward.
- **→ Rollout / Sample Adapter:** the trajectory it produces (the proxy-logged calls + the final reward) becomes the training sample.

Step-by-step example — one MBPP episode from the harness's view.

```
async def run_episode(task, policy, sandbox):          # one episode
    # 1. SEED: write the problem + the checker into the fresh sandbox
    for name, content in task.setup_files():           # e.g. PROBLEM.md
        await sandbox.write_file(name, content)
    await sandbox.write_file("run_checks.py", task.runner_source())

    messages = task.initial_messages()                 # system + PROBLEM.md
    solved = False
```

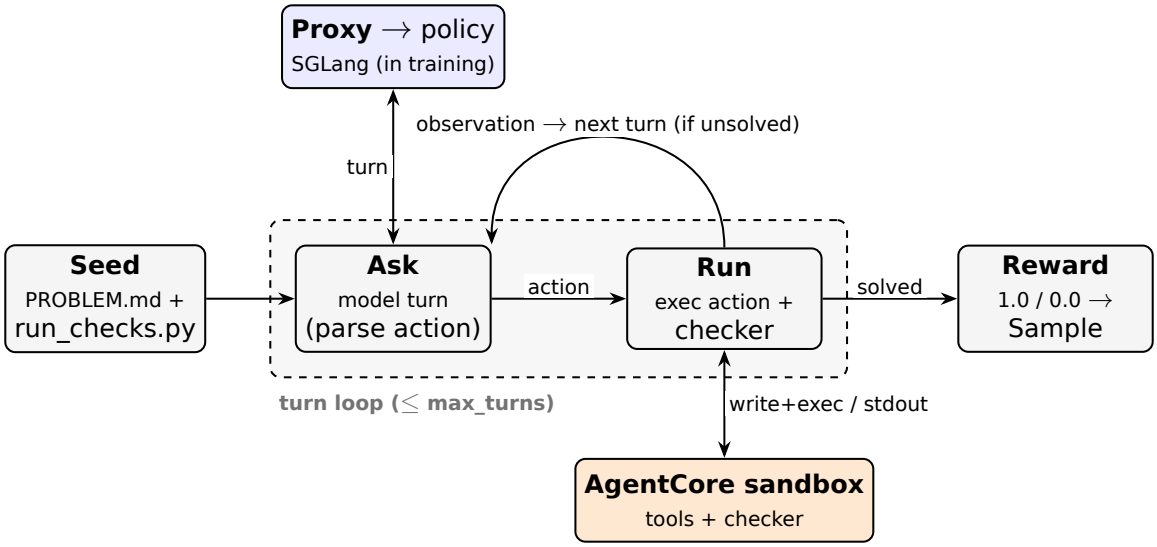


Figure 2: The harness's episode loop (`run_episode`). It **seeds** the sandbox once, then repeats a **turn loop**: ask the proxy for a model turn, parse the proposed action, run it (and the checker) in the AgentCore sandbox, and feed the observation back for the next turn. The loop exits when the checker passes or `max_turns` is reached; the final **reward** and the loss-masked trajectory become one training `Sample`. The proxy (policy under training) and the sandbox are the only two services the loop calls each turn.

```
    for _ in range(task.max_turns):                    # 2. TURN LOOP
        text = await policy.complete(messages)         #    (a) ask proxy -> policy
↪   turn
        code = task.extract_submission(text)           #    (b) parse the action
↪   (fenced code)
        await sandbox.write_file("solution.py", code)  #    (c) place the artifact
        exit_code, out, err = await sandbox.exec(       #    (d) run the checker IN
↪   the sandbox
            "python3 run_checks.py", timeout=60)
        if exit_code == 0:                              #    (e) checker passed?
            solved = True
            break
        messages.append(task.observation(out, err))    #    (f) feed the failure
↪   back, loop

    reward = 1.0 if solved else 0.0                    # 3. REWARD (from the checker)
    return build_token_stream(messages, reward)        # 4. loss-masked trajectory ->
     ↪  Sample
```

Walking it: the harness **seeds** the sandbox (step 1), then repeats **ask → parse → run → observe** (step 2a–f), each "ask" going through the proxy to the in-training policy and each "run" going to the AgentCore sandbox. When the checker exits `0` the episode is **solved**; otherwise the failure output is appended and the model tries again, up to `max_turns`. The final **reward** (step 3) comes from the checker — not a judge — and the harness hands the loss-masked trajectory to the training-data path (step 4). The harness owns the loop; the proxy and sandbox are the two services it calls each turn.

2.3 The proxy — every model call is the policy under training

What it is. The proxy is a thin CPU service that exposes a **standard LLM API** (in this project an Anthropic-Messages–compatible `AnthropicAdapter`, but an OpenAI-compatible endpoint works the same) and routes every request to the **in-training inference engine** (SGLang) instead of to any external provider. It is the single seam that lets an *off-the-shelf agent harness* — one written to call a normal LLM API — drive a model that is still being trained.

What it does. Three jobs, all on the critical path of every turn:

1. **Substitute the endpoint.** The harness thinks it is calling a hosted LLM; the proxy answers with the **policy under training**, holding the latest synced weights. No harness change is needed to point it at the trainer.

2. **Record the exact tokens.** For each call the proxy captures the **prompt token IDs and the sampled response token IDs** (and log-probs). The training engine only ever observes this stream of request/response pairs — so recording the *exact* sampled tokens is what avoids the retokenization/decode-drift problem in harnessed agentic RL [1].
3. **Bucket by episode.** Each request is tagged with the episode's `session_id`, so all calls of one rollout are grouped into the right trajectory before the Sample Adapter turns them into training samples.

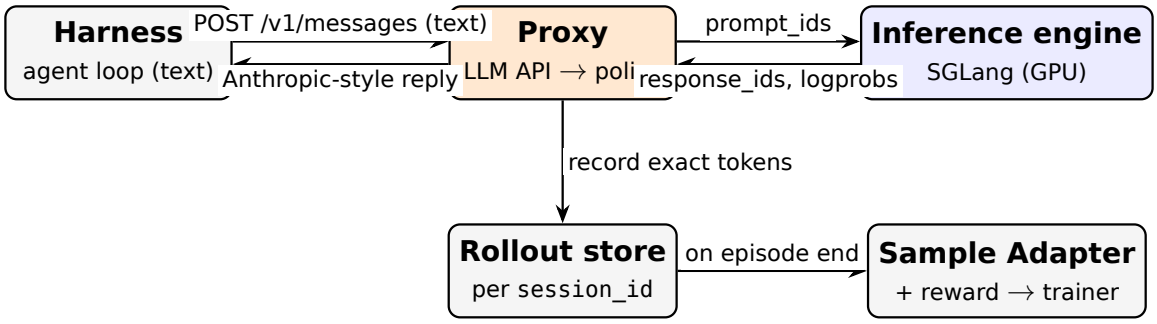


Figure 3: The proxy on the seam between the text-based harness and the token-based inference engine. It **substitutes** the endpoint (the harness's ordinary LLM API call is answered by the policy under training), and it **records** the exact prompt and response token IDs (with log-probs) into a per-`session_id` rollout store. On episode end the Sample Adapter reads that store, attaches the reward, and emits training samples — so the trainer learns from the exact tokens that were sampled — the *token-in-token-out* contract discussed next.

End-to-end example — one turn through the proxy. Consider a single MBPP coding turn:

```
1. Harness builds the chat and calls the proxy (drop-in LLM API):
     POST /v1/messages   { model: "policy",
                           messages: [system, PROBLEM.md, prior observations],
                           metadata: { session_id: "ep_9f3c⋯" } }

2. Proxy → Inference engine (SGLang, current policy weights):
     generate(prompt_ids) → response_ids, text, logprobs

3. Proxy records the on-policy call for training:
     rollout_store[session_id].append(
        { prompt_ids, response_ids, logprobs } )   # exact sampled tokens

4. Proxy returns an Anthropic-style response to the harness:
```

```
     { content: "```python\n def solve(): ···```", stop_reason: "end_turn" }

5. Harness extracts solution.py, runs it in the AgentCore sandbox,
   appends the checker's stdout/exit-code as the next observation, loops.

6. Episode ends → reward = 1.0/0.0. Sample Adapter reads
   rollout_store["ep_9f3c···"], attaches the reward/advantage, and emits
   loss-masked training samples (mask 0 on prompt/observation tokens,
   1 on the policy's own response tokens).
```

The key point: **without the proxy, the trainer could not observe — or take credit for — the tokens the harness actually sampled.** The proxy is simultaneously the *substitution* point (external API → policy) and the *observation* point (turns → training samples), which is exactly why the disaggregated design in Figure 1 puts it on the seam between the agent runtime and the training server.

2.4 Token-in-token-out (TITO) — the proxy's core responsibility

What it is. *Token-in-token-out* (TITO) is the contract that the rollout path carries **token IDs**, not just text: the inference engine returns the **exact token IDs it sampled** (plus their log-probs), and the trainer consumes **those same token IDs** for the policy-gradient update — never a sequence re-derived by tokenizing decoded text. In a disaggregated design the training engine never touches the environment; it only ever sees the request/response stream the proxy logged [1]. TITO is the guarantee that this stream is *token-exact*.

Why it matters — retokenization drift. A harness works in **text**: it sends messages and reads a string back. But a tokenizer is not a clean round-trip — decoding the sampled token IDs to a string and then **re-encoding** that string can yield a *different* token sequence than the one the model actually sampled (whitespace merges, byte-pair boundary shifts, special-token handling). If training re-tokenizes, three things silently misalign:

- **The action.** The token the loss is computed on is no longer the token the policy emitted.
- **The log-prob / importance ratio.** On-policy RL weights each token by the ratio of new to old policy probability; if the "old" tokens are reconstructed rather than recorded, the ratio is computed against tokens that were never sampled, biasing the gradient.

- **The loss mask.** The mask that marks *policy* tokens (train) vs. *prompt/observation* tokens (ignore) is defined per token; a shifted boundary leaks environment tokens into the loss, or drops the model's own.

For RLVR — where the reward is a precise 1.0/0.0 from a checker — this token-level slop quietly corrupts the very signal the method depends on. TITO removes it by construction.

Why the proxy owns it. The proxy is the **only** component that straddles both sides of the seam: it speaks the **text** LLM API to the harness and the **token** API to the inference engine (SGLang returns `response_ids` + log-probs). So it is the natural — and only correct — place to *capture the sampled token IDs at the moment of generation* and forward them intact, rather than letting a downstream step guess them from text. Concretely, the proxy records `{prompt_ids, response_ids, logprobs}` per turn (Figure above); the **Sample Adapter** then builds the training `Sample` directly from those IDs and the loss mask, and attaches the reward. The harness stays blissfully text-only, yet the trainer still gets token-exact, on-policy data. In short: **the proxy is where an off-the-shelf, text-based agent loop is reconciled with the token-exact requirement of RL — that reconciliation is TITO.**

2.5 slime — the trainer, and its asynchronous rollout

The trainer in this report is **slime** (THUDM) — an RL post-training framework built on **Megatron** (training) + **SGLang** (rollout), designed so that agentic workloads plug in as *data-generation* functions without forking the training kernel [9]. Its documented architecture maps cleanly onto Figure 1: a **Data Buffer** bridges a **Megatron** training group and one or more **SGLang** rollout servers behind a **router**; a *custom rollout generation* hook (where our harness + AgentCore sandbox live) produces samples into the buffer, and Megatron periodically pushes updated weights to the SGLang servers.

Asynchronous mode. By default training and rollout alternate, so the GPUs on each side idle while the other runs. slime avoids this with an **async training driver** (`train_async.py` instead of `train.py`): while Megatron trains on the current batch, the rollout side already generates the *next* rollout's data, overlapping the two [9]. For agentic RL — where rollouts have a **long tail** (one slow episode, e.g. a lengthy compile, can stall a whole step)

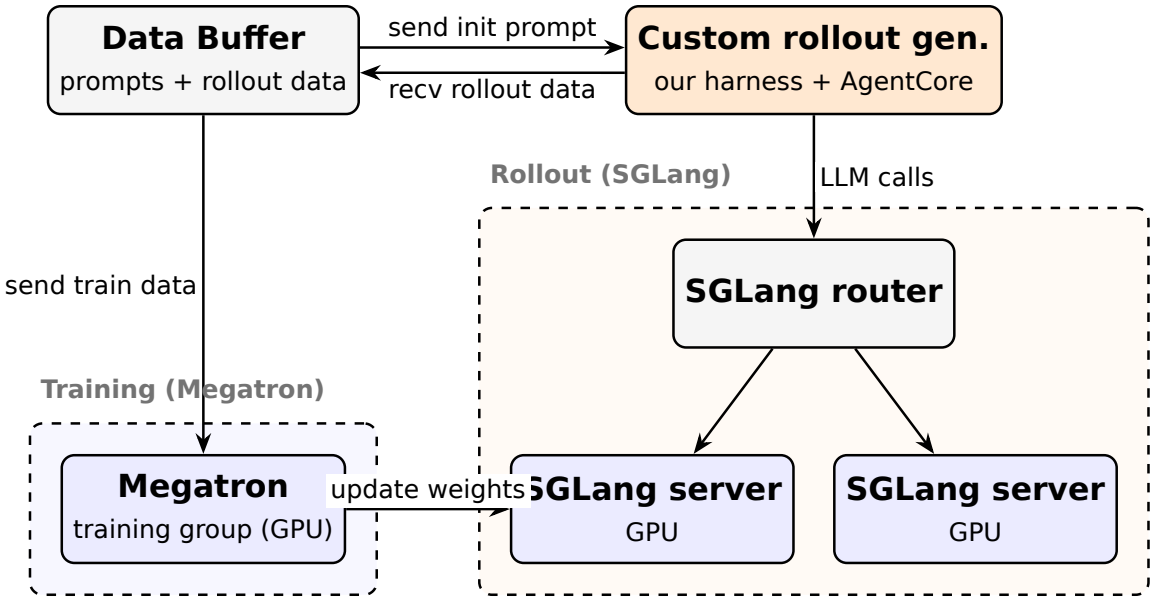


Figure 4: slime's architecture (redrawn after the slime documentation [9]). A **Data Buffer** bridges the two sides: it hands prompts to a *custom rollout generation* hook — where our **harness + AgentCore sandbox** live — and receives finished rollout data back. Generation runs through an **SGLang router** to one or more GPU **SGLang servers**. On the training side the buffer feeds **Megatron**, which periodically pushes **updated weights** to the SGLang servers. Agentic workloads plug into the hook without forking the training kernel.

— slime also offers a **fully-async rollout** path (`--rollout-function-path slime.rollout.fully_async_rollout.generate_rollout_fully_async`) that keeps a fixed pool of in-flight generations across rollout boundaries, so a training step never waits on the slowest sample. This is exactly the regime this report targets: **AgentCore sandboxes have variable, sometimes long, per-episode latency**, and async rollout keeps the trainer fed while they run.

Sources: slime repository https://github.com/THUDM/slime and documentation https://thudm.github.io/slime/ [9].

2.6 Scaling the sandbox — and why inference can become the bottleneck

Why scale the sandbox at all? A policy only *generalizes* if it is trained across many **diverse tasks** — SWE repos, SQL, math, tool-use, RAG, CTF, GPU kernels — and across **many episodes of each** so the reward signal is not dominated by one family. That means running a large number of **concurrent episodes** per training step, and every concurrent episode needs its own isolated sandbox. So scaling *up the number of concurrent sandboxes* (the harness concurrency knob, `CONC`) is the direct lever on task diversity and throughput.

But sandboxes do not run in isolation. Every one of those concurrent episodes still gets its model turns from the **same** inference engine. If we raise sandbox concurrency while holding the number (and size) of inference replicas **fixed**, generation requests start to **queue** at the inference engine: per-turn latency climbs, and the extra sandboxes sit idle waiting for their next turn. Past that point the inference engine — not the sandbox — is the bottleneck, and adding sandboxes buys nothing.

So first diagnose which side is slow. Per episode the wall-clock is split between (a) **generation** — the inference engine producing the turn — and (b) **sandbox execution** — running the tool call and the checker. The harness measures both: `[sglang-gen]` → `perf/sglang_gen_ms_{p50,p95}` for generation, and `[sbx-op]` → `perf/sandbox_ms_{p50,p95}` for the sandbox round-trip. Compare them before scaling:

- **Generation dominates** (`sglang_gen_ms` ≫ `sandbox_ms`) ⇒ **inference-bound**. Scale the inference engine (more replicas / more GPUs / larger tensor-parallel), *not* the sandbox count.

- **Sandbox dominates** (`sandbox_ms` ≫ `sglang_gen_ms`) ⇒ **sandbox-bound**. Scale sandbox concurrency / instances; one inference replica can feed many episodes.

Examples.

- **Sandbox-bound — KernelBench (GPU).** The model emits a short CUDA kernel in a few hundred ms, but `nvcc` compilation plus `allclose` on the GPU takes **seconds**. Sandbox time dwarfs generation → add GPU sandbox Instances; a single inference replica keeps up.
- **Inference-bound — math / long-reasoning.** The checker is a one-line stdlib assert (sandbox ≈ tens of ms), but the model generates a long chain of thought (generation ≈ seconds). Inference time dwarfs the sandbox → adding sandboxes does nothing; add inference replicas.
- **Balanced — SWE with a moderate test suite.** Generation and a multi-second test run are comparable → scale **both** roughly in proportion to keep either side from queuing.

The practical loop: pick a concurrency, read `perf/sglang_gen_ms_*` and `perf/sandbox_ms_*`, and grow whichever side dominates until the two are balanced — then raise both together.

Balancing load as you scale — let the router do inference, round-robin the sandbox pool. Once you add replicas on *either* side, the next question is how episodes get spread across them. The two sides answer it differently, and it matters that you *not* apply the same rule to both:

> **Do you need a load balancer?** For the **sandbox: no** within one Runtime — the session id *is* the balancer (the capacity provider places each session for you); across a *pool* of Runtime ARNs you add only trivial **sticky round-robin by session id**, not a real LB. For **inference: also no** — slime's one SGLang router already does cache-aware balancing; you just target its URL. In short: you never stand up a load balancer of your own; you mint distinct session ids, and (only across a Runtime pool) route them stickily.

- **Inference — do not hand-roll round-robin.** slime already runs **one SGLang router** in front of its inference replicas, and the router does **cache-aware (prefix-affinity) load balancing**: it steers each request to the replica most likely to already hold that prompt's prefix in its KV cache, falling back to shortest-queue only when caches are cold. That beats naive round-robin,

which would scatter a shared system-prompt/few-shot prefix across every replica and *destroy* the prefix cache — inflating time-to-first-token. So every proxy and policy simply targets the **router URL**; scaling inference is "add replicas," with zero client change and no load-balancing code of your own.

- **Sandbox — the session id *is* the load-balancing key.** One `runtimeSessionId` maps to exactly one isolated microVM / instance. Within a **single** Runtime, AgentCore's capacity provider already **places each new session on an available instance**, so simply minting a distinct per-episode session id spreads the load for you. You only need *explicit* round-robin when you run a **pool of sandbox Runtimes** (several ARNs). Three things trigger a pool — otherwise a single ARN is enough: **(1) concurrency** — your in-flight episodes (`S = CONC × num_engines`, or `N × S` once you replicate harness workers) would exceed one Runtime's per-region session cap, so you split the load across K ARNs; **(2) AZ/account spread** — placing Runtimes in different availability zones (resilience) or accounts (isolation, separate quotas); **(3) capability routing** — different families need *different sandbox images* (a GPU family, a DB-baked image, a per-repo SWE image, §3), so you register one ARN per capability and pick by `family` rather than pure round-robin. Below these thresholds — episode count fits one Runtime's quota and one generic image serves every family — you stay on a **single ARN** and let the capacity provider place sessions for you. When you do pool, pin each episode to one ARN and make it **sticky** so every op of that episode returns to the same warm sandbox that holds its workspace:

```python
import hashlib, os
from .generate import SGLangPolicy

# 1) Inference: target the ROUTER, never an individual replica. The router
#    load-balances (cache-aware, shortest-queue fallback) across all replicas,
#    so scaling inference is "add replicas" — no client-side balancing at all.
policy = SGLangPolicy(
    router_url=f"http://{args.sglang_router_ip}:{args.sglang_router_port}",
    model_path=args.hf_checkpoint,
)

# 2) Sandbox: within ONE Runtime the session id already spreads sessions across
#    instances. Across a POOL of sandbox Runtimes, round-robin BY session id so
#    the spread is even AND sticky (all ops of an episode hit one warm microVM).
SANDBOX_ARNS = os.environ["AGENTCORE_SANDBOX_RUNTIME_ARNS"].split(",")

def pick_sandbox_arn(session_id: str) -> str:
    h = int(hashlib.sha1(session_id.encode()).hexdigest(), 16)
    return SANDBOX_ARNS[h % len(SANDBOX_ARNS)]   # deterministic, sticky per episode
```

Hashing the session id (rather than a shared counter) gives an even, **sticky** spread with no cross-thread lock — which matters because episodes run concurrently on `asyncio.to_thread`. A plain counter (`itertools.cycle` behind a lock) is the simpler *literal* round-robin, but only use it if you do not need stickiness — and here you do: every op of an episode (seed → exec → read) must land on the microVM that holds that episode's files.

*(Implementation note: the shipped `make_sandbox()` (`sandbox.py`) actually uses the **`itertools.cycle`-behind-a-lock** counter, not the session-id hash sketched above. Stickiness still holds — the ARN is chosen **once** when the episode's sandbox object is created and fixed for all its ops — so each episode keeps one warm microVM; only the* selection* is a rotating counter rather than a hash. The hash form above is the lock-free alternative you would switch to if the shared counter's lock became a contention point at high `N × S`.)*

2.7 Scaling the harness too — the persistent driver behind both dials

The previous subsection balanced the two *remote* dials, inference and sandbox. But both are **driven** by a third component that is easy to forget because it is neither on a GPU nor remote: the **harness** — the agent loop that opens each episode's sandbox, calls the policy turn by turn, runs the checker, and packs the result into a `Sample`. It has its own throughput ceiling, and if you scale inference and sandbox to fill a deep async buffer while leaving the harness alone, the bottleneck simply **moves onto the driver**: the buffer-fill rate is `min(inference, sandbox, harness)`, and adding replicas to two of the three buys nothing once the third saturates.

Today the harness is a single process. slime runs **one** `RolloutManager` Ray actor whose single event loop drives *every* `generate` coroutine (`slime/ray/rollout.py`); the number of concurrent episodes is bounded by one `asyncio.Semaphore` sized `S = CONC × num_engines` — with the demos' `CONC=4` and two SGLang engines, `S = 8` in flight (`slime/rollout/sglang_rollout.py`). There is **no slime knob for "number of rollout worker processes"**: `CONC` raises concurrency *within* the one process, not the number of processes. A second, finer semaphore (`INFER_CONC`, default `0` = off; `policy.py`) can independently cap concurrent SGLang POSTs, but it too lives inside that single process.

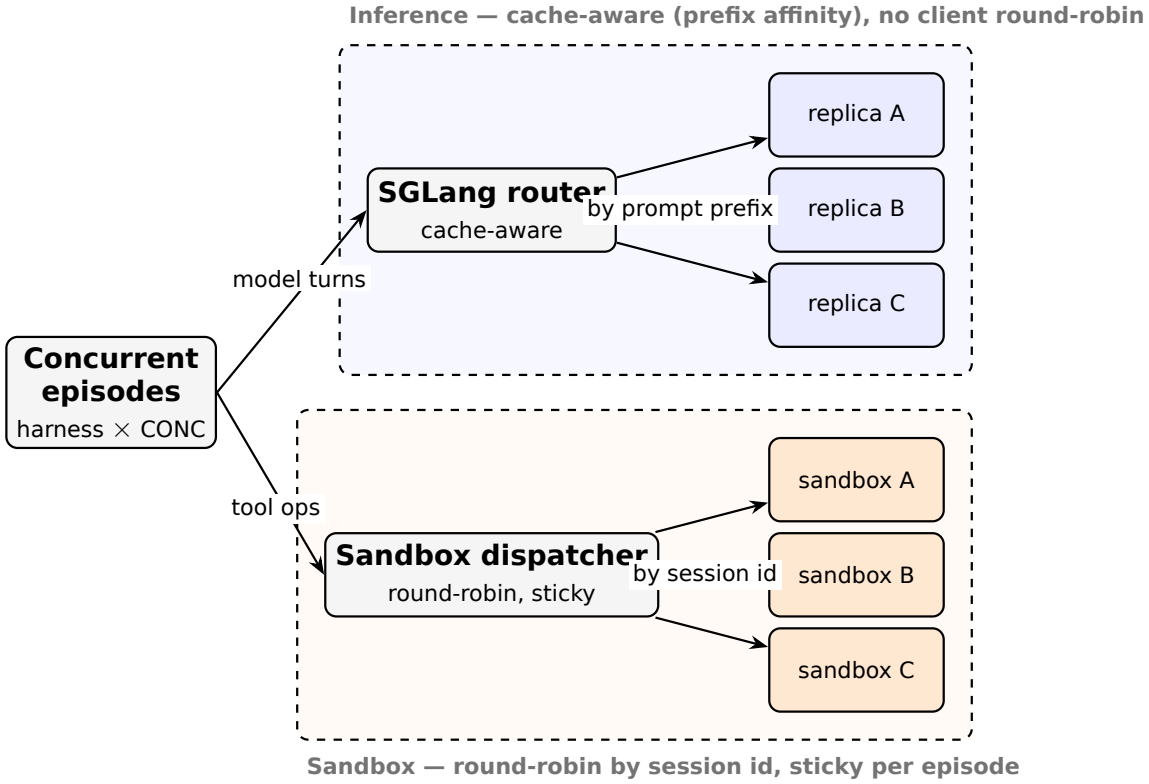


Figure 5: Load balancing differs by side. **Inference (top):** every episode's model turns go to *one* SGLang router, which does **cache-aware** balancing — it steers a request to the replica most likely to already hold its *prompt prefix* in the KV cache (shortest-queue fallback), so a shared system/few-shot prefix is not scattered. The client never round-robins; it just targets the router URL, and scaling is "add replicas." **Sandbox (bottom):** one `runtimeSessionId` = one isolated microVM/instance. Within a single Runtime the capacity provider spreads sessions for you; across a *pool* of sandbox Runtimes you **round-robin by session id**, kept **sticky** so every op of an episode (seed → exec → read) returns to the same warm microVM that holds its files.

Why one process ceilings. Concurrency here is `asyncio`, so it overlaps *I/O waits* — but not CPU work, which the Python **GIL** serializes: every turn JSON-encodes an invoke payload, decodes the response, and (at episode end) builds the loss-masked token stream, and those run one-at-a-time. The synchronous boto3 `invoke_agent_runtime` is offloaded with `asyncio.to_thread` onto the **default, unconfigured** executor (≈ `min(32, cpu+4)` threads; `agentcore_runtime_sandbox.py`), and each episode builds its own boto3 client at botocore's default **10-connection** pool (`max_pool_connections` is never tuned). Every in-flight episode also holds open file descriptors / sockets and buffered trajectory state in RAM. So past some `S`, raising `CONC` just deepens internal queues: per-episode latency climbs and the buffer stops filling faster. This project **flags but has not measured** that ceiling — a `run.sh` comment notes raising `CONC` "also raises client-side load (asyncio thread pool, boto3 clients)," and the chosen `CONC=4` is documented as "a guess, not a measured optimum" (`docs/sandbox-concurrency-ablation.md`).

The asymmetry that decides how you scale it. The sandbox is **ad-hoc**: one microVM is created and torn down *per episode*. The harness is the opposite — a **persistent, warm** process that holds the event loop, the SGLang/proxy connection, the boto3 clients, and the trajectory buffers *across* episodes (it needs warmth, not isolation, precisely because episodes are independent — §"The harness"). So "scale the harness" does **not** mean "make it ad-hoc like the sandbox." It means **replicate warm worker processes**: run `N` rollout processes, each driving `S` concurrent episodes, for `N × S` in flight. The sandbox multiplies by *launching more disposable microVMs*; the harness multiplies by *running more long-lived drivers*.

Component	Lifetime / isolation	Scaling unit	Ceiling of one unit	Where it runs
Inference (SGLang)	persistent / shared	replica behind the router	GPU KV cache + decode throughput	trainer GPUs
Harness	**persistent, warm / none needed**	**worker process × `CONC`**	**GIL, `to_thread` pool, FDs, RAM**	**trainer box, or a CPU fleet**

Component	Lifetime / isolation	Scaling unit	Ceiling of one unit	Where it runs
Sandbox (Agent-Core)	ad-hoc, per episode / strong (microVM)	session id; pooled Runtime ARNs	per-Runtime session cap	AgentCore microVM / Instance

Where to put the harness — three options.

- **Co-located on the trainer box (today's default).** The one `RolloutManager` runs next to Megatron and SGLang. Cheapest and lowest-latency to the buffer, and fine at $S \approx 8$; but its threads, FDs, and RAM compete with the trainer for the box's CPU budget, so it contends as you push S up.
- **Scaled out to a CPU rollout fleet (disaggregated).** Because slime drives generation from a single actor, going past one process is an *architecture* change, not a flag flip: run several rollout drivers (shard prompts across CPU nodes, each executing the same `generate` hook) that dial the **shared** SGLang router and the **pooled** sandbox ARNs. This decouples harness CPU/FDs/RAM from the GPU box and lets you scale drivers independently — the disaggregated-placement lever of §"Open problems" applied to the harness rather than to the trainer/inference split.
- **Hosted on an AgentCore Runtime.** AgentCore *can* run the agent loop as a persistent CPU Runtime (this project keeps that path as untested scaffolding, `agentcore_runtime_generate.py`). A pool of long-lived harness Runtimes is an elastic option, but it adds a network hop between the harness and slime's buffer and spends isolation the harness does not need — which is why the report defaults to in-process.

The joint recipe. First diagnose the harness the way §"Scaling the sandbox" diagnoses the other two: raise `CONC` and watch whether **samples-per-second / buffer depth** (both instrumented, §"The shared skeleton") keep climbing — if they flatten while the harness host's CPU is pegged (or you hit "too many open files"), the driver, not the GPUs, is the limit. Then grow the three in balance: inference replicas behind the cache-aware router, sandbox Runtimes round-robined sticky-per-episode, **and** harness workers (`N` processes × S). A Little's-law sizing keeps them matched: the in-flight episodes needed to saturate inference ≈ `inference throughput × mean`

`episode latency`; provision `N × S` to hit that number, then raise all three together until GPU **goodput** — not any single dial — stops improving.

3 Tasks frontier labs run in a sandbox

Frontier models are post-trained with **reinforcement learning against executable, verifiable environments** (RLVR). The common denominator is a **sandbox seam**: the model's untrusted output runs in an isolated environment, and a **programmatic verifier** — not a human, and increasingly not an LLM judge — turns the outcome into a scalar reward.

3.1 Two verifier styles — verifiable and reference-based

Two verifier styles recur across the open reports:

- **Verifiable / rule-based** — a deterministic check: run generated code against hidden unit tests, verify a math answer by symbolic equivalence, compare a SQL result-set, or validate a captured flag. The reward is typically **binary** (all tests pass → 1, else 0) and cannot be gamed by an eloquent-but-wrong answer.
- **Reference-based model judge** — an LLM (or reward model) scores the response against a ground-truth reference. Used where outputs are open-ended (science explanations, instruction following, chat), often alongside rule-based rewards.

3.2 What the frontier reports describe

The trend is toward **verifiable rewards at scale**: Kimi K2, for example, runs a rollout infrastructure of **more than 10,000 concurrent execution sandboxes** on Kubernetes for its coding and software-engineering RL [5]. What the major open reports describe:

Model	RL / verifier setup	Task domains	Reward
DeepSeek-R1 [2]	GRPO; rule-based verifier + code compiler (no neural reward model, to avoid reward hacking)	math, competitive code	answer match / tests pass (binary)
Qwen3 [3]	Reasoning RL from **3,995 query–verifier pairs**; General RL over **20+ tasks**; multi-turn tool use with real-environment feedback	math, code, tool-use, RAG, instruction	rule-based verifiable + reference-based model judge + reward model
Llama-Nemotron [4]	GRPO (science) + RLOO (instruction following); vLLM + Megatron; prompts difficulty-filtered by pass-rate	science reasoning, math, instruction	**LLM-judge** (Llama-3.3-70B) answer-match + `<think>` format reward
AceReason-Nemotron [10]	math-then-code RL; a **local sandbox verifier** runs code against *all* test cases (64-worker pool)	math (~49k problems), code (~8.5k problems)	binary: pass **all** tests / `sympy` symbolic match on `\boxed{}`

Model	RL / verifier setup	Task domains	Reward
Kimi K2 [5]	agentic RL; **verifiable-rewards "gym" + a tool simulator (world model) + real execution sandboxes** (>10k concurrent)	agentic tool use, coding, SWE, math, instruction	executable unit tests + verifiable checks + self-critique rubric

NVIDIA also **publishes the post-training data mix** openly: the Nemotron post-training datasets [12] quantify the domain balance used to train an open reasoning model — roughly **Math 239k, Code 175k, STEM 355k, Chat 628k** samples (plus large multilingual splits) — a concrete picture of how much of each family a frontier post-training run consumes.

3.3 Task families in detail

Each family below lists its **dataset, reward function, key metric, and environment requirement** — the properties that determine what a sandbox must provide — plus an **episode shape**: the typical *turns per episode*, the *response length per turn* (the policy's own tokens), and the resulting *total episode length* (all turns' responses plus the observations fed back). The episode shape is what you size the training config against: it sets **`--rollout-max-response-len`** (per-turn generation budget), the **max turns** cap (`EPISODE_MAX_TURNS`, default **3** in this project's demos), the **sequence-length / context window** the trainer and inference engine must hold, and the **rollout-buffer** memory per sample. The numbers below are typical order-of-magnitude ranges (reasoning models sit at the high end of every response figure), not hard limits — measure your own before committing a config.

3.3.1 Single-turn, generation-bound families

The checker is milliseconds; nearly all the wall-clock is one long generation. These are the *inference-bound* families — scale the SGLang side.

- **Math & logical reasoning** — *Dataset:* GSM8K, MATH-500 [2], AIME / HMMT; **DeepScaleR + NuminaMath** for RL (~49k problems) [10]. *Reward:* **verifiable answer match** — e.g. a `sympy`-based check of symbolic equivalence on the `\boxed{}` answer, binary, with no format or length bonus [10]. *Metric:* accuracy, pass@k (Qwen3-235B's AIME'24 rose **70.1 → 85.1** over 170 RL steps [3]). *Env:* a lightweight **CPU checker** (milliseconds). *Episode shape:* **almost always single-turn** — there is no environment feedback loop, just one answer. But that one response is a **long reasoning trace: ~1k–8k tokens, and up to 16k–32k** for hard competition problems on a reasoning model. Episode length ≈ that one response; strongly **inference-bound** (the checker is a one-line assert).
- **Code generation & competitive programming** — *Dataset:* MBPP [8], HumanEval, **LiveCodeBench** [11]; competitive sets from **Codeforces, AtCoder, LeetCode** [10]. *Reward:* **hidden unit tests pass** — AceReason grants reward *only if the code passes every test case* [10]. *Metric:* pass@1 / pass@k, Codeforces ELO, LiveCodeBench (Kimi K2 v6: **53.7%** [5]). *Env:* a language interpreter plus a **test-runner sandbox with per-case time limits**, run at scale (AceReason: a 64-worker process pool [10]; Kimi K2: >10k concurrent [5]). *Episode shape:* **few turns — often 1, up to ~5** with retry on failed tests. The submitted program is short (**~100–800 tokens**), but a *reasoning* model prepends a long chain-of-thought, pushing the per-turn response to **several thousand tokens**; total episode is typically **~2k–16k tokens**, generation-heavy rather than context-heavy.
- **Instruction following** — *Dataset:* synthetic prompts carrying **one to ten verifiable instructions** (Llama-Nemotron's RLOO stage [4]; part of Qwen3's 20+-task General RL [3]). *Reward:* **deterministic verification via a code interpreter** for the checkable constraints, plus a **format reward**, with an LLM judge for the open-ended remainder [3, 4]. *Metric:* IFEval-style constraint satisfaction. *Env:* a **code interpreter** to evaluate the constraints. *Episode shape:* **typically single-turn** — one response scored against the constraints. Response length tracks the asked-for output, **~200 tokens–2k**; episode ≈ that one response.

3.3.2 Iterative, few-turn families

A short retry loop against a checker — write, run, read the failure, fix. One family (GPU kernels) is where the *checker* dominates cost.

- **GPU kernels / systems** — *Dataset:* KernelBench [7]. *Reward:* numerical **`allclose`** vs. a reference **plus a speedup target**. *Metric:* correctness rate, speedup. *Env:* an **NVIDIA GPU + CUDA toolchain** (`nvcc`) — the one family that requires a GPU, not just CPU. *Episode shape:* **a few turns ($_{1\text{–}5}$), iterating on compile errors and speedup. Each response is one CUDA kernel, 200–1,000 tokens**; total episode a **few thousand tokens**. Cost lives in the *sandbox* (`nvcc` + `allclose` take seconds), not in generation — the canonical sandbox-bound family.
- **SQL / data analysis** — *Dataset:* text-to-SQL suites (Spider-style). *Reward:* **result-set equality** against gold rows (the query is executed against a real engine). *Metric:* execution accuracy. *Env:* a local SQL engine (e.g. SQLite), self-contained. *Episode shape:* **1–3 turns** (write query; retry on a SQL error or empty result-set). Response is a single query, **~30–300 tokens**; episode is **small (a few hundred to ~2k tokens)** — cheap on both sides.

3.3.3 Multi-turn, observation-heavy families

The response each turn is short, but the sandbox feeds a large observation (tool result, retrieved passages, command output) back every turn — cost moves into *context growth and the sandbox*.

- **Tool use / function calling** — *Dataset:* live and **simulated** tool environments; Kimi K2 builds a repository of **3,000+ real MCP tools plus 20,000+ synthetic tools** and generates multi-turn trajectories against a **tool simulator that acts as a world model** [5]; Qwen3 trains with real-environment execution feedback [3]. *Reward:* **correct final result *and* the right tools invoked** (verifiable), sometimes with a model judge. *Metric:* task success on agentic tool-use benchmarks (Kimi K2: **τ^2-Bench 66.1, ACEBench 76.5** [5]). *Env:* tool stubs / an API sandbox / a **stateful simulator** that returns realistic feedback across turns. *Episode shape:* **multi-turn, one turn per tool call — typically ~3–10+ turns**. The policy's response each turn is short (a tool call plus brief reasoning, **~50–300 tokens**), but the *observations* (tool results)

can be large; total episode is **~2k–10k tokens**, dominated by fed-back tool output rather than by generation.

- **RAG & search agents** — *Dataset:* open-domain QA over a corpus or live web search (part of Qwen3's General RL, which explicitly targets RAG [3]). *Reward:* **answer grounded in retrieved facts** — Kimi K2 adds a **sentence-level faithfulness judge** [5]. *Metric:* exact-match / F1. *Env:* a retrieval index or search API plus **network egress**. *Episode shape:* **multi-turn — ~2–8 turns** (issue query → read passages → refine or answer). The policy's responses (queries, final answer) are short, but each retrieved-document observation is large, so the **episode context is retrieval-heavy — commonly 8k–32k+ tokens** even though the model generates comparatively little.
- **Security / CTF** — *Dataset:* CTF challenge sets. *Reward:* a **checker validates the captured flag**. *Metric:* solve rate. *Env:* an isolated shell, sometimes with network to a remote challenge service. *Episode shape:* **multi-turn exploration — ~5–30+ turns** of shell commands. Per-turn responses are short (a command plus reasoning, **~50–400 tokens**); observations (command output) vary from a line to a hex dump, so the **episode ranges from a few thousand to tens of thousands of tokens**.

3.3.4 Long-horizon, session-bound families

Tens to hundreds of turns; the accumulated transcript is the largest of all, and the session must stay alive for the whole workflow — the families that most need a warm, long-lived sandbox.

- **Software engineering (SWE)** — *Dataset:* SWE-bench / SWE-bench Verified [6]; Kimi K2 additionally mines **executable unit tests from real GitHub pull requests and issues** [5]. *Reward:* the repo's **test suite turns green** (`FAIL_TO_PASS` + `PASS_TO_PASS`), binary. *Metric:* % issues resolved (Kimi K2 reports **65.8%** single-attempt on SWE-bench Verified [5]). *Env:* a per-repository toolchain — usually a **prebuilt Docker image** carrying the exact dependencies, run in isolation per task. *Episode shape:* **long-horizon and multi-turn** — real SWE agents take **~15–50+ turns** (locate → edit → run tests → read failure → re-edit); each turn's response is a command or a **diff/patch of ~100–1,000 tokens**, but the observations (file contents, test logs) are large, so the **episode context routinely reaches tens of thousands of tokens (16k–128k+)** — the most demanding family for context length.

- **Web / computer use** — *Dataset:* WebArena-style browser tasks. *Reward:* **goal state reached** (a DOM / state checker). *Metric:* task success. *Env:* a real **browser / rendering engine** and network. *Episode shape:* **long-horizon — ~10–50+ turns**, one UI action (click/type/scroll) per turn. Each response is tiny (**~20–150 tokens**), but every observation (a DOM dump or accessibility tree, sometimes a screenshot) is very large, making this — with SWE — the **heaviest family for episode context (tens of thousands of tokens, growing per turn)**.
- **Multi-agent / long-horizon** — *Dataset:* cooperative or long-workflow tasks. *Reward:* a **verifiable end state**. *Metric:* task success. *Env:* many turns / long wall-clock, with state persisted across a long session. *Episode shape:* **the longest horizon — tens to hundreds of turns**, per-turn responses short-to-moderate, but the accumulated transcript makes the **episode the largest of all (often 32k–128k+ tokens)** and demands a session that stays alive for the whole workflow.

3.4 Reading the episode shape into a config

Two axes drive every knob. **Turns per episode** sets `EPISODE_MAX_TURNS` and, multiplied by the per-turn response, the *sequence length* the trainer and inference engine must hold: single-turn families (math, instruction following) need only a long `--rollout-max-response-len`; long-horizon ones (SWE, web/computer use, multi-agent) need a large **context window** and a **session that stays warm for the whole episode**. **Where the tokens are** sets which side you scale (§ Scaling the sandbox): generation-heavy families (math, reasoning code) are *inference-bound* — long responses, trivial checker; observation-heavy families (SWE, RAG, web, tool use) push cost into *context growth and the sandbox*. Rule of thumb: budget context ≈ `max_turns × (response_per_turn + observation_per_turn)`, size `--rollout-max-response-len` to the per-turn response at the high end (reasoning traces), and set rollout-buffer memory from the total episode length so long-horizon samples do not overflow it.

3.5 What every family shares

Regardless of domain, each needs the same two things: an **isolated environment** to execute untrusted model output, and a **programmatic checker** to score it —

the sandbox seam. Where the reports differ is only in *what runs inside* (a Python interpreter, a CUDA toolchain, a browser, a tool simulator, a full repo image) and *how the checker scores* (tests pass, symbolic match, result-set equality, flag validation, or — for open-ended work — a reference-based judge). The rest of this report turns to running these workloads on a concrete sandbox platform.

3.6 Language and environment per family — and how one microVM serves them all

The families above differ sharply in *what must be installed to run and check them*. The table names, per family, the **language the model's artifact is written in**, the **environment dependencies the checker needs**, and whether it fits the **CPU microVM** or requires the **GPU Instances** substrate. This is the concrete "what runs inside" that the sandbox image (or a per-session install) must provide.

Family (§3)	Dataset(s)	Artifact language	Environment / dependencies the checker needs	Substrate
Math & logical reasoning	GSM8K, MATH-500, DeepScaleR	Python (answer string)	Python stdlib + `sympy` (symbolic/numeric match)	CPU
Code gen & competitive	MBPP, HumanEval, LiveCodeBench	Python (competitive: often C++)	Python interpreter + test runner (`unittest`/asserts) + per-case timeout	CPU
Instruction following	synthetic (IFEval-style)	text / Python	code interpreter (stdlib) to verify each constraint	CPU

Family (§3)	Dataset(s)	Artifact language	Environment / dependencies the checker needs	Substrate
GPU kernels / systems	KernelBench	**CUDA C++** + Python driver	**NVIDIA GPU + CUDA toolkit (`nvcc`) + PyTorch** (`allclose`, speedup)	**GPU**
SQL / data analysis	Spider-style, Gretel text-to-SQL	SQL (+ Python harness)	a SQL engine — `sqlite3` (in the Python stdlib)	CPU
Tool use / function calling	xLAM, ToolACE	Python / JSON tool calls	Python + tool stubs / API sandbox / a **stateful simulator**; MCP via Gateway	CPU
RAG & search agents	HotpotQA, open-domain QA	Python (queries + answer)	a retrieval index / search API + **network egress**	CPU
Security / CTF	CTF challenge sets	shell + C / Python / binaries	an isolated shell + common tooling, sometimes **network** to a challenge service	CPU

Family (§3)	Dataset(s)	Artifact language	Environment / dependencies the checker needs	Substrate
Software engineering (SWE)	SWE-bench / Verified	Python (repo language varies)	**per-repo toolchain** — pinned deps + system packages, `pytest`; a large image	CPU (amd64)
Web / computer use	WebArena-style	Python + browser actions	a **headless browser / rendering engine** + network	CPU
Multi-agent / long-horizon	cooperative / long-workflow	varies	the sub-tasks' deps + a **long-lived session** holding state	CPU

How can one microVM support such different environments? The key is that **the microVM does not need one universal image** — it reconciles the variation three ways, in increasing order of how structural the difference is (this is the §"One image per Runtime, but a different env per row" logic applied to *languages/deps* rather than to per-repo images):

1. **Task-agnostic sandbox → most differences are just *files*, not images.** The sandbox exposes only `write_file` / `exec` / `read_file`, and every session is an **isolated microVM** booting *your* OCI image. All task specificity — the prompt, seed files, and the hidden checker — is *written into that episode's session at seed time*, not baked into the image. So a **single generic Python image** already serves the CPU families that need nothing beyond the interpreter and stdlib: math (`sympy`), code (`unittest`), SQL (`sqlite3` is stdlib), instruction

following, tool-use stubs, and light CTF. A math episode, a SQL episode, and a tool-use episode are just *different files* in three identical, isolated microVMs.

2. **Per-session runtime install → heavier or unusual deps go into the ephemeral workspace.** This has two variants with *different* implementation status. **(a) Deferred to `CMD`** — the sandbox image installs a large dep when the container boots, before the server binds, keeping the *built* image small; this is **implemented** and is how the GPU sandbox installs the ~2.7 GB CUDA torch wheel to stay under the 2 GB image cap (`deploy/Dockerfile.sandbox.gpu`). **(b) Installed at *seed time*** — the harness would `git clone` / `pip install` / `apt` a family's deps into that session's workspace on turn 0. This is a **described design, not yet implemented**: the shipped demos seed only small files (`PROBLEM.md`, `run_checks.py`, `solution.py`), and the seed-time-install path is the SWE workaround the report explicitly flags as *"a design answer, not a measured one"* (§"One image per Runtime, but a different env per row"). Either way, because each `runtimeSessionId` is a **separate microVM**, one episode's installed environment **never leaks into another's** — the isolation is what would make per-session customization safe.
3. **Per-capability image → a *pool* of Runtimes when the difference is structural.** Some differences cannot be installed at runtime: a **GPU toolchain** (KernelBench needs `nvcc` + a GPU — the **Instances/amd64** substrate, not the CPU microVM), a *different interpreter version*, root `apt` the base image can't grant, or a multi-GB repo image. For those you register **one Runtime ARN per image**. What is **implemented** today is the pool mechanism as a *same-image throughput fan-out*: `make_sandbox()` round-robins episodes across `AGENTCORE_SANDBOX_RUNTIME_ARNS` (`sandbox.py`, an `itertools.cycle`), assuming the ARNs are interchangeable. Routing an episode to a *different* image **by `family`** — true capability routing — is a **described design, not implemented**: the selector never keys on the task family, and the `family` discriminator in `generate.py` only chooses the *Task subclass* (prompt + checker), not the image/ARN. This is why the family table's **Substrate** column matters: everything marked *CPU* can share one generic microVM image (differences absorbed by #1 and #2a); only the *GPU* row forces a second image on a different substrate — and that CPU↔GPU split is selected by env var per run, not by per-episode family routing.

In short: **one microVM = one image per session; task diversity is absorbed inside a single generic image (#1) plus, where wired, a `CMD`-time install (#2a), and only genuine image/substrate differences — chiefly the GPU**

family — need a second Runtime (#3). The heavier reconciliations — seed-time installs (#2b) and per-family image routing (#3) — are the report's design for SWE/mixed-image workloads, not paths the current demos exercise. Task diversity is absorbed inside one microVM image; only capability diversity multiplies images.

3.7 The one family that stresses the image: SWE and the microVM's arm64 + 2 GB limits

Most families above run inside a small, generic image — a Python interpreter and a stdlib checker fit comfortably. **Software engineering (SWE)** is the exception, and it is worth calling out before the platform section because it collides with two properties of AgentCore's *managed serverless microVM* — and the two collide with each other:

- **arm64-only (documented).** The managed microVM runs **only arm64** images: the official Runtime service contract states the *"Platform: ARM64 container — Required for compatibility with the AgentCore Runtime environment"* [D17] (the GPU capacity-provider substrate, by contrast, is amd64 — §"Building a customized Runtime"). A per-repository SWE image must therefore be **cross-built to arm64**.
- **A tight built-image ceiling.** The managed microVM image must stay small — this project measured a **~2 GB (2048 MB)** ceiling on the built image, which is exactly why its GPU sandbox installs the CUDA torch wheel at `CMD` rather than baking it in (§"Building a customized Runtime"). A realistic SWE-bench repo image — the exact compiler, system packages, and pinned dependencies a project needs — routinely exceeds a couple of GB.

The two interact badly for SWE specifically, because **SWE-bench's own tooling treats arm64 as second-class**: its README states plainly that *"Support for `arm64` machines is experimental"*, and on ARM hosts it does not ship prebuilt images — you must pass `--task-repo` so *"the images are built locally with Docker Buildx"* [6b]. So the very images you would reuse are (a) only *experimentally* available on the architecture the microVM forces, and (b) sized for x86 developer boxes, not a 2 GB serverless cap. The heaviest-image family lands on the substrate least suited to it.

Workarounds — in rough order of how well they fit this project's architecture:

1. **Fetch the repo and deps at *seed* time, not in the image.** The sandbox is task-agnostic (`write_file` / `exec` / `read_file`), so the harness can `git clone` the target repo and `pip install`/build its dependencies into the *session workspace* on the first turn, instead of shipping them in the OCI image. This moves the repo bulk off the image entirely and into the per-session ephemeral filesystem (good for 8 h) — sidestepping the image cap. Cost: per-episode setup latency and **network egress**, so the Runtime needs public egress or VPC interface endpoints (§"Network isolation"; [D11]).
2. **Defer heavy installs to `CMD`.** Keep the *built* image lean and `pip`/`apt` the large layers when the session boots — the exact trick the GPU sandbox already uses for torch (§"Building a customized Runtime"). Cost: added cold-start (the GPU demo pays ~90 s for its torch install, §`demo_gpu_single`).
3. **Slim the repo image.** Multi-stage builds, `--no-install-recommends`, drop build toolchains and package caches after compiling, and start from a slim arm64 base — standard Docker hygiene, often enough to bring a repo image under the cap.
4. **Cross-build to arm64 with Buildx, and *validate the checker there*.** `docker buildx build --platform linux/arm64`; for SWE-bench use its `--task-repo` local-build path. Because arm64 is experimental upstream, re-run the `FAIL_TO_PASS`/`PASS_TO_PASS` checker on arm64 before trusting the reward — an arch-specific test flake would silently corrupt the RLVR signal.
5. **Move SWE to the amd64 GPU/Instances substrate.** Instances run **amd64**, allow far larger images, and add a **persistent EBS volume** (so a one-time repo/deps install survives stop/resume) and **14-day** sessions (§"Two compute types"). This sidesteps both the arch and the image cap at once; the cost is spending an EC2 instance (and its GPU) that a CPU-only SWE checker does not otherwise need.

This is the concrete reason the §"Can AgentCore host every frontier sandbox?" table marks the per-repo Docker path with a ◑ rather than a clean ✓: for SWE, the sandbox *image* — not the checker or the GPU — is the binding constraint.

3.8 One image per Runtime, but a different env per row: how AgentCore reconciles it

SWE-bench raises a sharper version of the same problem: **each task instance ships its own Docker environment** (a specific interpreter version, system pack-

ages, and pinned dependencies checked out at one commit) — hundreds of distinct images across the benchmark. AgentCore's binding constraint is that a **Runtime ARN is bound to exactly one container image**, chosen once at `CreateAgentRuntime` (`containerConfiguration.containerUri`, an ECR image — §"Building a customized Runtime"). An `invoke_agent_runtime` call passes only a **`runtimeSessionId` + payload — never an image [D8]**, so you **cannot swap the Docker environment per invocation**. Three ways reconcile "one image per ARN" with "a different env per row" — and the practical answer is usually a hybrid:

- **A — Move the per-row difference *out of the image, into per-session setup* (this project's native model).** The sandbox is task-agnostic (`write_file` / `exec` / `read_file`) and all task specificity already lives in files the harness writes into *that episode's session* at seed time. Extend that: on turn 0 the harness runs `git clone` + `git checkout <base_commit>`, installs the pinned deps, and drops in the hidden `run_checks.py` — so "a different image per row" becomes "a different **setup script** per row, run inside one generic image." Each `runtimeSessionId` is an isolated microVM, so row *i*'s installed env never leaks into row *j*. *Cost:* per-episode setup latency + **network egress** (public or VPC endpoints, [D11]). *Limit:* works when the delta is *installable at runtime*; it struggles when a row needs a **different interpreter version**, **root apt** the base image can't grant, or a heavy compiled toolchain — those belong in the image.
- **B — A *pool* of Runtimes, one image per ARN, routed by row.** Register **K Runtime ARNs, each with a different baked image**, and have the harness pick the ARN by the row's repo/family instead of round-robin (the *capability-routed* dispatch described in §"Demos"). But hundreds of per-instance images cannot each get a Runtime (operationally heavy, per-region limits), so group at **repo-family** granularity (one `django` image, one `sympy` image, …), **not** per-instance.
- **C — The practical hybrid (family base image + per-row delta).** Bake the *expensive, shared* layer per repo-family ARN (interpreter version, system libs, repo at a common base) → B; install the *cheap per-commit delta* (that instance's pip deps) at seed → A. This keeps each image small — under the microVM's ~2 GB arm64 cap above — while one ARN still covers many instances. If images are simply too large or too x86-bound, fall back to the amd64 **Instances** substrate (workaround #5).

Caveat: this is a **design answer, not a measured one** — SWE is the one family this

project has **not** yet wired to a real dataset or run on hardware (the real-dataset work covers GSM8K/MATH-500/HumanEval; SWE-bench/SQL/RAG remain TODO), and, per above, SWE-bench's arm64 support is itself experimental.

4 Foundations of AgentCore Runtime

Amazon Bedrock AgentCore [D1] is a family of managed services for running and securing agents and tools — any framework, any model, fully serverless. This report uses one of them, **AgentCore Runtime**, as the execution **sandbox** for RL. This section defines its three core concepts, its two compute types, how AgentCore manages their lifecycle, and how you build and deploy a customized Runtime — cross-checking the official documentation [D1–D13, D17] against AWS's own SDK and sample code [D14–D16].

4.1 Runtime, session, and microVM — three concepts, precisely

These three words are used loosely elsewhere; the integration depends on keeping them distinct.

- **Runtime (agent runtime).** A *versioned, containerized host* you create with `CreateAgentRuntime` [D9]. It packages your code — a container image in ECR, or a zipped Python bundle — behind an ARN. Creating a Runtime auto-creates version **V1** and a **DEFAULT endpoint**; you can publish further versions and custom endpoints and route invocations to them. The Runtime is the *deployable unit*; it holds no per-user state of its own.
- **Session.** One logical task or conversation, addressed by a **`runtimeSessionId`** you pass on `InvokeAgentRuntime` [D8] (client-supplied or Runtime-generated; **minimum 33 characters**, max 256, carried in the `X-Amzn-Bedrock-AgentCore-Runtime-Session-Id` header). The **first** invoke for a new id **cold-starts** dedicated compute; every **subsequent** invoke with the same id **re-routes to that same warm compute**, so files, memory, and processes persist *across turns within an episode*. Different ids are fully isolated.
- **microVM.** The *unit of isolation* backing a session on the default compute: a lightweight virtual machine with its **own CPU, memory, and filesystem**, sanitized on termination. One session maps to one microVM; a microVM is never

shared across sessions. Its container contract [D17]: an **ARM64** image listening on **`0.0.0.0:8080`** that serves **`POST /invocations`** (the work) and **`GET /ping`** (health, returning `{"status": "Healthy"}` or `"HealthyBusy"`) — the same contract AWS's SDK scaffolds for you [D15].

The cardinality. These three nest **one-to-many-to-one**: **one Runtime serves many concurrent sessions, and each session maps to exactly one microVM (1 : N : 1)**. The Runtime is a stateless *template* (one image, one ARN); every distinct `runtimeSessionId` you invoke against it mints its own isolated microVM, and AgentCore's capacity provider places each one for you — so the session id, not a second Runtime, is how you get a second sandbox. You create *additional* Runtimes only when the **image**, the per-region **session cap**, or the **AZ/account** must differ (§"Scaling the sandbox"); otherwise a single Runtime backs all your concurrent sessions. Put another way: **the number of Runtimes tracks the number of distinct sandbox *images* (capabilities), not the number of tasks** — because the sandbox is task-agnostic (`write_file` / `exec` / `read_file`), one generic image serves every task family that runs on the same substrate, so even a mixed-family training run stays on **one Runtime, many sessions** (§`demo_cpu_mixtask`).

In RL terms: **the Runtime is the deployable sandbox image, the session is one episode, and the microVM is the isolated box that episode executes in.** So *N* concurrent episodes = *N* session ids on one Runtime — not *N* Runtimes.

4.2 microVM vs a Docker container vs E2B

Because "sandbox" and "container" are often used interchangeably, it is worth being precise about what the microVM actually is — especially since we are running **untrusted, model-generated code**, where the strength of the isolation boundary is the whole point.

- **A Docker container** isolates via Linux namespaces and cgroups but **shares the host kernel**. That is fine for packaging and for trusted workloads, but a shared kernel is a comparatively soft boundary for hostile code: a kernel exploit escapes to the host and to co-tenants. Containers alone are not a safe multi-tenant boundary for arbitrary generated code.
- **An AgentCore microVM** is a lightweight, **hardware-virtualized** virtual machine (a Firecracker-class microVM) with its **own guest kernel**, its own CPU/memory/filesystem, and a VM-strength boundary — while still booting

your OCI container image inside it. So it is not "container *or* VM": you keep container packaging *and* gain VM isolation, provisioned per session and sanitized on termination. On top of that it is **AWS-native** — reached only by in-account SigV4/IAM, placeable in your VPC, and integrated with the other AgentCore services (Gateway, Memory, Identity, Observability) — and it can carry a **GPU** (via Instances) and **long sessions** (8 h microVM / 14 d Instances).

- **E2B** is an independent sandbox platform that, like AgentCore, runs code in **Firecracker microVMs** (its templates are also defined from Dockerfiles), so the *isolation model is comparable*. The differences are operational, not architectural: E2B is its own cloud (or self-hosted) service with its own SDK and identity, oriented to short-lived code-interpreter sandboxes, whereas AgentCore is a managed AWS service that runs **in your own account** under IAM/VPC, adds **GPU on Instances** and **multi-hour/multi-day sessions**, and slots into the rest of the Bedrock AgentCore stack.

	Docker container	AgentCore microVM	E2B sandbox
Isolation boundary	shared host kernel (namespaces/cgroups)	**own kernel** — Firecracker-class microVM	own kernel — Firecracker microVM
Runs your image?	yes (OCI)	yes — your OCI image *inside* a microVM	Dockerfile-defined templates
Managed by	you / your orchestrator	**AWS, in your account** (IAM/SigV4/VPC)	E2B service (cloud or self-host)
GPU	host-dependent	**via Instances** (EC2)	not the core offering
Session length	you manage	8 h microVM / 14 d Instances	short-lived (code-interpreter)

For an RL sandbox running untrusted policy output, the microVM's per-session hardware isolation is the property that matters: one episode's generated code cannot see another episode's files, the trainer's weights, or the host — and the box is destroyed and its memory wiped when the session ends.

4.3 Two compute types — managed serverless (microVM) vs Instances (GPU EC2)

AgentCore Runtime offers two substrates, chosen at create time; the loop code is identical across them.

- **Serverless microVM (default).** No servers to manage: AgentCore provisions a per-session microVM on demand. **CPU only** — a microVM has *no GPU* [D5]. Sessions last up to **8 hours** with a **15-minute idle timeout** (reset on each invoke). This fits the large majority of RL sandboxes: code execution + unit tests, math/SQL checkers, tool simulators.
- **Instances (GPU).** When a task needs a GPU, Runtime runs on the **Instances** compute type — real **EC2 instances in your own account**, provisioned through a **capacity provider** (`capacityProviderConfiguration` → `ec2Configuration`) [D5, D9]. Supported GPU families include **g4dn, g5, g6, g6e, gr6, g6f, gr6f, g7e** (plus `inf2`); AgentCore provisions the NVIDIA drivers, so a standard CUDA image works unmodified. Sessions can run up to **14 days**, a **persistent EBS volume** survives a session stop and re-attaches on resume, and — unlike the microVM — one instance/session can host **many agents** (1:N). Crucially, one `runtimeSessionId` still maps to exactly **one instance = one GPU**, so there is **no cross-session GPU time-slicing**: you scale GPUs by adding sessions/instances, not by fractioning one.

> **Can you pick a CPU-only EC2 instance for an Instance?** Not as a documented path. The `ec2Configuration` families AWS lists [D5, D9] are all **accelerated** (g4dn/g5/g6/g6e/gr6/g6f/gr6f/g7e + `inf2`) — there is **no general-purpose (m/c/t) CPU family**. The split is deliberate: **CPU-only work belongs on the serverless microVM**, and the Instances substrate exists to add a **GPU**. So if you want a larger/x86/longer-lived *CPU* box (e.g. to escape the microVM's arm64 + 2 GB image limits, §3), today's only route is to launch a GPU-family Instance and leave the GPU idle — you pay for an accelerator you don't use. (The supported list is per the docs as of this work; re-check `ec2Configuration` if a plain-CPU Instance later appears.)

Property	Serverless microVM	Instances (GPU)
Compute	managed, per session	EC2 in your account (capacity provider)
GPU	none [D5]	g4dn/g5/g6/g6e/g7e… + inf2
Max session	8 h	14 d
Isolation unit	1 microVM / session	1 instance / session (1 GPU)
Filesystem	ephemeral	persistent EBS across stop/resume
Agents / session	1	many (1:N)

4.4 How AgentCore manages the lifecycle

Runtime lifecycle (versions & endpoints). `CreateAgentRuntime` builds version V1 and the DEFAULT endpoint; updates publish new versions; you route traffic by endpoint. Deleting the Runtime removes the deployable — sessions are not part of the Runtime's own state.

Session lifecycle (the part that matters for RL). A session moves through **Active → Idle → Terminated** [D3, D4]:

- **Launch / cold start.** The first invoke with a fresh `runtimeSessionId` provisions the microVM (or, on Instances, may cold-launch an EC2 instance — *minutes* for a GPU box). Same-id invokes thereafter hit warm compute.
- **Active / Idle.** Each invoke resets the **15-minute idle timer**; between invokes the session sits Idle but warm.
- **Termination.** Automatic on **idle expiry (15 min)**, **max lifetime** (microVM **8 h** / Instances **14 d**), or a failed health check → the microVM is **terminated and its memory sanitized**. You can also end a session explicitly with **`StopRuntimeSession`** [D12]. On Instances, the EBS volume persists across a stop.

Latency — measure, don't assume. The docs promise "fast cold starts" but publish no number [D2]; a GPU first-invoke can take **minutes** to launch

the EC2. This project therefore instruments every sandbox round-trip (`[sbx-op]` → `perf/sandbox_ms_{p50,p95}`) and every generation (`[sglang-gen]` → `perf/sglang_gen_ms_*`) — the same signals the scaling analysis in §“Scaling the sandbox” depends on.

4.5 Session lifecycle in the RL loop — per-episode sessions, and why row count ≠ instance count

Two consequences of the lifecycle above drive how a rollout consumes sessions, and both surprise people sizing a run.

One episode = one session (= one instance on GPU). The generate hook mints a fresh `runtimeSessionId` per episode — *“one session id == one isolated sandbox for the whole episode”* — so the session id is the isolation unit. On the serverless substrate that is one microVM; on Instances it is one EC2 instance = **one GPU**, a strict 1:1 with **no cross-session GPU time-slicing** (§“Two compute types”).

Terminate: explicit vs idle-reclaim — and this project’s choice per backend. A session can be ended explicitly (**`StopRuntimeSession`** [D12], or the Code-Interpreter client’s `stop`) or simply abandoned and left to the idle timer. The three sandbox backends deliberately differ at episode end (`__aexit__`):

Backend (`SANDBOX=`)	At episode end	Session ends by
`agentcore` (Code Interpreter microVM)	calls the client `stop()`	**explicit**, immediately
`agentcore-runtime` (sandbox-as-Runtime)	nothing — just drops the client	**idle reclaim** (15-min microVM / 600 s GPU-instance timeout) or max lifetime
`agentcore-ray-farm` (B4)	sends `op=close` for the sub_session	frees the **fractional-GPU actor** immediately; the shared g6 session persists

The plain-runtime choice to rely on idle reclaim is intentional: a warm instance is reused by the next episode’s session (round-robin across the K-Runtime pool), so churning short-lived episodes benefits from **not** tearing the instance down between

them. `StopRuntimeSession` is there if a deployment needs to reclaim capacity sooner.

Row count ≠ instance count. A 100-row dataset (KernelBench level_1) does **not** launch 100 instances. Rows stream through episodes across training steps; at any instant only `--sglang-server-concurrency` (default **4**) episodes are in flight, so the live-instance count = concurrent sessions ≈ CONC, and those few instances **recycle through all rows** over time. What multiplies instances is *concurrency*, not data:

- **Demo B (`agentcore-runtime`, `demo_gpu_single`):** one session per episode × 1:1 ⇒ **N concurrent episodes = N concurrent g6 instances** (bounded by CONC and the capacity provider's max-instance cap).
- **Demo B4 (`agentcore-ray-farm`, `demo_gpu_shared`):** all concurrent episodes share **one** `runtimeSessionId` → **one** g6 instance; each episode's sub_session leases a fractional-GPU Ray actor, so N concurrent episodes still = one instance.

This is the concrete cost lever: to raise sandbox concurrency on a GPU task **without** paying for N instances, pack episodes onto one instance (B4) rather than fanning out sessions (B).

4.6 Building a customized Runtime

There are three ways to put your code on a Runtime, in increasing order of control:

1. **The `agentcore` CLI (fastest).** AWS ships an official CLI; the newer, **recommended** one is the **`@aws/agentcore`** npm package [D16] (the earlier Python `bedrock-agentcore-starter-toolkit` is the legacy path). It offers `agentcore create`, `agentcore dev` (local loop), `agentcore deploy`, and `agentcore invoke`. Its default **CodeZip** build packages your Python without Docker; the **Container** build builds an image from **your own Dockerfile** (Docker required).
2. **Bring your own container (what this project does).** Author a container satisfying the contract (ARM64; `0.0.0.0:8080`; `/invocations` + `/ping` [D17]) and register it with `CreateAgentRuntime`'s **`containerConfiguration.containerUri`** (an ECR image). This is the maximum-control path — you own the base image, system packages, and toolchain, exactly what

a task-specific sandbox (a CUDA stack, a full repo image) needs. `CreateAgentRuntime` takes a **union**: `containerConfiguration` (ECR image) **or** `codeConfiguration` (zip) — pick one [D9].

3. **The Python SDK.** The **`bedrock-agentcore`** SDK [D15] provides `BedrockAgentCoreApp` and the **`@app.entrypoint`** decorator, which implements the `/invocations` + `/ping` contract for you: you write one function, the SDK serves it. This project's sandbox server is exactly that (see §5).

The container contract, concretely. A minimal CPU sandbox image:

```
FROM public.ecr.aws/docker/library/python:3.12-slim
RUN pip install --no-cache-dir bedrock-agentcore
COPY agentcore_slime_demo/sandbox_server_agent.py /app/
ENV SANDBOX_WORKDIR=/workspace
CMD ["python", "-m", "agentcore_slime_demo.sandbox_server_agent"]   #
↪  BedrockAgentCoreApp serves :8080
```

A **GPU** sandbox changes two things: AgentCore injects `libcuda.so.1` (point LD_LIBRARY_PATH at it), and the CUDA torch wheel is large — so install it **at `CMD`, not baked into the image**, keeping the built image under AgentCore's **2048 MB** cap:

```
ENV LD_LIBRARY_PATH=/usr/lib64          # AgentCore injects libcuda.so.1 here
CMD ["sh","-c","pip install torch==2.5.1 --index-url
↪  https://download.pytorch.org/whl/cu124 \
               && exec python -m agentcore_slime_demo.sandbox_server_agent"]
```

Deploy. Managed microVM images must be **arm64**; a GPU capacity provider runs **amd64** — so cross-build. This project's one driver does the whole sequence:

```
export AWS_REGION=us-west-2
python3 deploy/deploy_two_runtime.py all   # ECR create → docker buildx --platform →
↪  push
                                           # → CreateAgentRuntime
                                           ↪  (containerConfiguration)
                                           # → (GPU) CreateCapacityProvider + attach
```

For GPU it additionally calls **`CreateCapacityProvider`** (ec2Configuration: `g6.xlarge`, VPC subnets/SGs, EBS, `idleInstanceTimeout=600`, `maxLifetime=86400`) and attaches it via `capacityProviderConfiguration` [D5, D9]. **IAM:** an execution role (ECR pull, logs, `InvokeAgentRuntime`) plus, for Instances, the operator policy `BedrockAgentCoreRuntimeInstancesOperatorRolePolicy`.

4.7 Example AWS repositories (use cases)

To ground all of the above against AWS's own code, three official repositories are worth reading:

- **AgentCore samples** [D14] — `awslabs/agentcore-samples`, the official example catalog, organized as `00-getting-started`, `01-features` (per-service examples, including the Runtime feature), `02-use-cases` (e.g. coding assistants, workflow-automation agents, a data-analyst assistant), and `04-infrastructure-as-code` (CloudFormation/CDK/Terraform). Read `01-features` and `04-infrastructure-as-code` for an end-to-end `CreateAgentRuntime` deploy in full.
- **AgentCore Python SDK** [D15] — `aws/bedrock-agentcore-sdk-python` (PyPI `bedrock-agentcore`): `BedrockAgentCoreApp` + `@app.entrypoint`, and the runtime service contract this project's sandbox server implements.
- **AgentCore CLI** [D16] — the newer `@aws/agentcore` (npm) `agentcore` command (create/dev/deploy/invoke), recommended over the legacy `bedrock-agentcore-starter-toolkit` for the zero-Docker path.

5 Using AgentCore as the sandbox in an agentic RL workflow

With the foundations in place, this section shows how AgentCore Runtime plugs into a real RL trainer — **slime** — as the execution **sandbox**, with the harness kept **in-process**. The integration is deliberately small: **no fork of slime**, two hook flags, and a sandbox client that turns each tool call into one `InvokeAgentRuntime`.

5.1 Where AgentCore attaches to slime — hooks, not a fork

slime [9] runs Megatron (training) and SGLang (rollout) asynchronously (§"slime — the trainer"). It exposes the rollout path as a **custom generation function**, so an agentic workload plugs in without touching the training kernel:

- `--custom-generate-function-path agentcore_slime_demo.generate.generate` — the per-prompt rollout: it drives the in-process harness, and each tool call goes to an AgentCore sandbox.

- `--custom-rm-path` — an optional custom reward hook (here the reward comes straight from the in-sandbox checker, so the generate hook attaches it directly).

The key design choice: **AgentCore is the sandbox, not the harness.** The harness (agent loop) is independent per episode and benefits from *warmth*, so it stays in-process next to the trainer; the sandbox needs *isolation* per episode, which is exactly AgentCore's per-session microVM. AgentCore *can* also host the harness (this project keeps that as untested scaffolding, `agentcore_runtime_generate.py`), but the recommended path is in-process.

5.2 The call path — one diagram

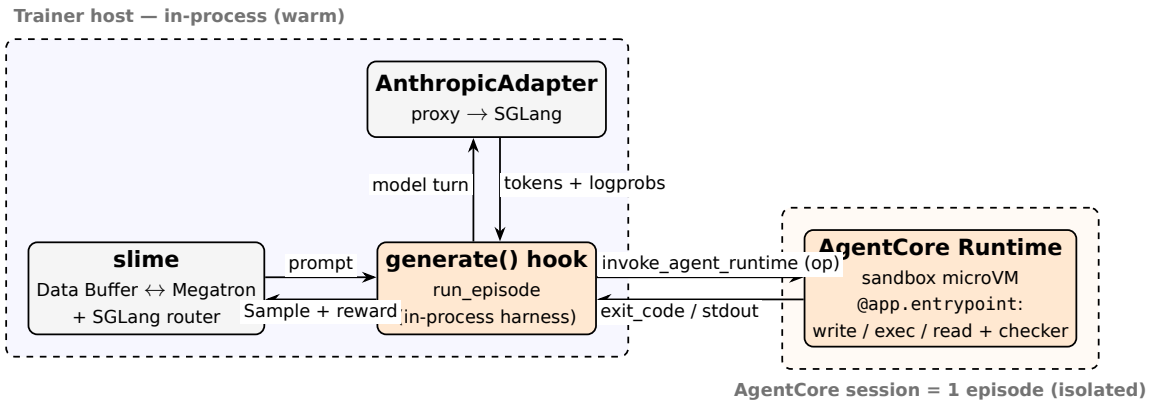


Figure 6: How AgentCore drops into slime. slime hands a **prompt** to the `generate()` hook, which runs the agent loop *in-process*. Each **model turn** goes through the **proxy** (which returns the exact sampled **tokens + logprobs** — the TITO property of §"Token-in-token-out"); each **tool call** becomes one SigV4 `invoke_agent_runtime` to the **AgentCore sandbox**, where `@app.entrypoint` runs the op and the **checker** (the reward source). The finished, loss-masked **Sample + reward** returns to slime's buffer. Everything but the sandbox is co-located on the trainer host; the sandbox is an isolated per-episode microVM.

5.3 Key code — three pieces

1. The generate hook (in-process rollout). slime calls this once per prompt. It builds the policy pointed at the SGLang **router** (so inference stays cache-aware-

balanced, §"Scaling the sandbox"), opens **one sandbox per episode**, runs the episode, and returns a single loss-masked `Sample`:

```
# agentcore_slime_demo/generate.py  (wired via --custom-generate-function-path)
async def generate(args, sample, sampling_params, evaluation=False):
    policy  = SGLangPol-
↪   icy(router_url=f"http://{args.sglang_router_ip}:{args.sglang_router_port}",
                          model_path=args.hf_checkpoint,
                          chat_template_kwargs={"enable_thinking": False})
    task    = _task_from_sample(sample)                       # rebuild Task from row
↪   metadata
    sandbox = make_sandbox(os.environ.get("SANDBOX", "agentcore"), region=...)
    async with sandbox as sb:                                  # one session == one
    ↪   episode
        traj = await run_episode(policy, sb, task, sampling_params)
    tokens, resp_len, loss_mask, rollout_logprobs = build_token_stream(traj,
↪   tokenizer)
    return Sample(tokens=tokens, response_length=resp_len, reward=traj.reward,
                  loss_mask=loss_mask, rollout_log_probs=rollout_logprobs,
                  metadata={"task_id": task.task_id, "solved": traj.solved,
                            "num_turns": traj.num_turns, "sandbox_id": sb.sandbox_id})
```

Two RL-critical details: the loss mask is **1 on the policy's own tokens, 0 on prompt/observation/tool tokens**, and `build_token_stream` reuses the **recorded `token_ids` per turn** — no re-tokenization, the TITO property of §"Token-in-token-out". It returns a *bare* `Sample` (not a list): slime's fully-async path chokes on nested sample lists.

2. The sandbox client (one op = one invoke). Each `write_file` / `exec` / `read_file` becomes a single SigV4 `invoke_agent_runtime`, keyed by the episode's session id. boto3 is synchronous, so it is offloaded with `asyncio.to_thread`, letting N episodes run concurrently:

```
# agentcore_slime_demo/agentcore_runtime_sandbox.py
def _invoke_sync(self, payload: dict) -> dict:
    resp = self._client.invoke_agent_runtime(                  # boto3
↪   "bedrock-agentcore"
        agentRuntimeArn=self._arn,
        runtimeSessionId=self._session_id,                     # >=33 chars; one per
↪   episode
        payload=json.dumps(payload).encode())
    return json.loads(resp["response"].read().decode())

async def exec(self, cmd, *, timeout=120, **kw):
    res = await asyncio.to_thread(self._invoke_sync,
```

```
            {"op": "exec", "command": cmd, "timeout": int(timeout), "workdir":
↪  self._workdir})
    return int(res.get("exit_code", 1)), res.get("stdout", ""), res.get("stderr", "")
```

The real client also **retries** transient 5xx/throttle blips with backoff (one bad round-trip must not crash a multi-hour run) and carries a much larger budget for GPU "insufficient capacity" while a g6 launches.

3. The Runtime server (what runs inside the sandbox). On the Runtime, the `bedrock-agentcore` SDK's `@app.entrypoint` receives each payload and dispatches on `op`. This is the whole sandbox server — and *the checker runs here, so the microVM is the reward source*:

```python
# agentcore_slime_demo/sandbox_server_agent.py  (deployed as the Runtime image)
from bedrock_agentcore.runtime import BedrockAgentCoreApp

def handle(request: dict) -> dict:              # pure dispatch (unit-testable
↪  without AgentCore)
    op = (request or {}).get("op", "ping")
    if op == "ping":       _ensure_workdir(request.get("workdir")); return {"ok":
    ↪  True}
    if op == "write_file": return _do_write(request)     # -> {"ok": True}
    if op == "read_file":  return _do_read(request)      # -> {"text": ...}
    if op == "exec":       return _do_exec(request)      # -> {exit_code, stdout,
    ↪  stderr}  (runs the checker)

app = BedrockAgentCoreApp()

@app.entrypoint
def entrypoint(request):
    return handle(request)

app.run()      # serves 0.0.0.0:8080 — POST /invocations + GET /ping
```

5.4 Who controls the AgentCore lifecycle — deploy driver vs. the hook

A precise point that the code above implies but is worth stating outright: **slime's training kernel controls neither the Runtime nor the session.** Lifecycle is split across two planes, and slime's core (`RolloutManager`, Data Buffer, Megatron, the SGLang router) is entirely **AgentCore-unaware** — it only calls the function registered at `--custom-generate-function-path` and gets a `Sample` back.

- **Runtime lifecycle (create/delete the ARN) — outside slime, at deploy time.** No slime component calls `CreateAgentRuntime`/`DeleteAgentRuntime`. The Runtime (and, for GPU, the capacity provider) is provisioned **once before training** by the deploy driver (`deploy/deploy_two_runtime.py`; for GPU, `deploy/redeploy_gpu_sandbox.py`), and slime is simply handed the ARN via `AGENTCORE_SANDBOX_RUNTIME_ARN`. The sandbox client only *reads* that env var and raises if it is unset — it never creates the Runtime. The Runtime is a **deploy-time artifact**, not a training-loop object.
- **Session lifecycle (launch/terminate per episode) — the custom `generate` hook and its sandbox client.** All session control lives *below* slime's seam, inside `agentcore_slime_demo.generate.generate`, which opens one sandbox per prompt (`async with make_sandbox(...) as sb:`). The client (`AgentCoreRuntimeSandbox`) mints **one `runtimeSessionId` per episode**, and:
  - **Launch is implicit** — there is no "create session" API. `__aenter__` sends the **first `invoke_agent_runtime`** (a `ping`) under the fresh session id, and that first invoke is what **cold-starts the microVM** (and, on GPU, warms the g6).
  - **Each op** (`write_file` / `exec` / `read_file`) is one more `invoke_agent_runtime` under the *same* session id.
  - **Termination is also implicit** — `__aexit__` does **not** call `StopRuntimeSession`; it just drops the client and lets AgentCore reclaim the session on its **~15-minute idle timeout**. slime never explicitly terminates a session on the happy path.

Action	API	Controlled by	When
Create / delete Runtime (+ capacity provider)	`CreateAgentRuntime` / `Delete⋯` / `CreateCapacityProvider`	**deploy driver** — *not slime*	once, before/after the run
Drive rollout	—	slime `RolloutManager` / Data Buffer — **AgentCore-unaware**	per prompt

Action	API	Controlled by	When
Launch session (cold-start microVM)	first `invoke_agent_runtime` (`ping`)	the hook's sandbox client `__aenter__`	per episode
Run ops	`invoke_agent_runtime` (write/exec/read)	sandbox client, same session id	each turn
Terminate session	*(implicit)* idle timeout; `StopRuntimeSession` exists but is unused	AgentCore reclaims it; `__aexit__` only drops the client	on ~15-min idle
Defer termination across weight-sync (optional)	repeated `ping`	`SessionKeeper`, *outside* the harness	during the sync barrier

Because termination is **passive** (idle reclamation, not a slime call), the weight-sync pause can trip the idle clock and lose a warm session — which is exactly the failure mode `demo_cpu_quiesce`'s `SessionKeeper` heartbeat addresses (§`demo_cpu_quiesce`). **One-sentence version:** the deploy driver owns the Runtime, the `generate` hook + its sandbox client own the session (launch on first invoke, terminate by idle timeout), and slime's training kernel owns neither.

5.5 One harness, many task families

The loop inside `run_episode` (`harness.py`) is **task-agnostic** — the same seed → turn → check → reward cycle serves every family; only the *prompt, seed files, and checker* change (`task_families.py`), dispatched by `metadata.task.family`:

```
for name, content in task.setup_files():                    # seed the workspace
    await sandbox.write_file(name, content)
await sandbox.write_file("run_checks.py", task.runner_source())
for _ in range(task.max_turns):                             # EPISODE_MAX_TURNS
↪  (default 3)
    text = await policy.complete(messages)                  # model turn (via proxy)
    await sandbox.write_file(task.artifact_filename, task.extract_submission(text))
    exit_code, out, err = await sandbox.exec("python3 run_checks.py", timeout=60)
```

```
    if exit_code == 0: solved = True; break
    messages.append(task.observation(exit_code, out, err)) # feed failure back
reward = task.reward(0 if solved else 1)                   # binary 1.0 / 0.0, from
↪  the in-sandbox checker
```

Datasets are built once and seeded per episode: each row carries `metadata.task` (family + spec), reference answers are dropped so they cannot leak into prompts, and `generate.py` rebuilds the `Task` from that metadata. The same code runs the checker for MBPP, SWE, SQL, math, tool-use, RAG, CTF, and the rest — the sandbox is the only thing that changes.

5.6 The episode, end to end

Taking one MBPP episode:

1. `run.sh` sets `SANDBOX=agentcore`, `FAMILY`, `CONC`, `EPISODE_MAX_TURNS`, then `ray job submit ⋯ train_async.py --custom-generate-function-path ⋯generate.generate`.
2. slime hands a prompt to `generate()`, which rebuilds the Task and opens `make_sandbox("agentcore")` → **cold-start** the session's microVM.
3. **Seed:** `write_file` `PROBLEM.md` + `run_checks.py`.
4. **Turn loop** (≤ `EPISODE_MAX_TURNS`): `policy.complete` via the proxy (`[sglang-gen]`) → extract `solution.py` → `exec("python3 run_checks.py")` (`[sbx-op]`) → on failure, feed the observation back.
5. `exit_code == 0` → solved; **reward = 1.0 / 0.0** from the in-sandbox checker.
6. `build_token_stream` → loss-masked `Sample` → buffer → Megatron step → weight sync to SGLang.

CPU → GPU is one env var: `SANDBOX=agentcore` (microVM, MBPP) → a GPU-Runtime ARN (Instances, KernelBench, `allclose` on cuda) — **zero change to the loop**.

5.7 Scaling, sharing, and network isolation

- **Scaling** follows §"Scaling the sandbox": grow the side that dominates the per-op latencies, and balance by letting the SGLang router handle inference (cache-aware) while round-robining a pool of sandbox Runtimes (sticky per episode).

- **Sharing a GPU.** One session = one GPU, with no cross-session time-slicing [D5]. To pack several episodes onto one physical GPU you either run **many agents in one session** (they share the instance — no isolation) or run **your own fractional scheduler** (this project's `RayGpuFarm`, `num_gpus=0.5` actors, with an `nvcc flock` because a non-MIG L4 wedges under compile storms). Isolated scaling means *more instances*; there is **no MIG on L4**.
- **Network isolation — pull, don't expose.** Every Runtime is *reached* by an outbound SigV4 `invoke_agent_runtime`; **no Runtime has an inbound port**. IAM/SigV4 is the auth; the `session_id` is routing, not auth. For `networkMode=VPC` the Runtime has no internet route, so it needs interface endpoints (`ecr.api`, `ecr.dkr`, `s3`, `logs`, `sts`, `bedrock-agentcore`), and the ENI must land in an AZ that serves `bedrock-agentcore` — miss one and create fails with a null reason [D10, D11].

5.8 Can AgentCore host every frontier sandbox?

Mapping the task families of §"Tasks frontier labs run in a sandbox" onto AgentCore, the only hard limit is GPU **fractioning** — not any missing capability (✓ native ·◑ needs a BYO container / egress):

Task / sandbox	✓?	How — and the limit
Code exec + unit tests (DeepSeek, Qwen) [2,3]	✓	Code Interpreter, or your image on a Runtime
Math / logic verifier	✓	stdlib checker in the CPU microVM
Tool-use env (Kimi K2) [5]	✓	Python tools; MCP via Gateway

Task / sandbox	✓?	How — and the limit
Per-repo Docker image (SWE) [6]	◑	Runtime runs your OCI image, isolated per session — but the managed microVM is **arm64-only** and caps the image at **2 GB** (§3); slim it, defer installs to `CMD`, or use the amd64 Instances substrate
Browser / computer use	✓	managed Browser tool
Networked RAG / web read	◑	egress — public, or VPC + endpoints [D11]
GPU kernel / CUDA (KernelBench) [7]	◑	Instances only; 1 session = 1 GPU; **no MIG on L4**

In this project the policy is always **Qwen3-4B**, trained by slime, with the harness in-process and AgentCore as the sandbox; what varies from demo to demo is only the **sandbox substrate** (CPU microVM vs GPU Runtime instance) and the **task**. The set (detailed in §“Demos”) is **sixteen** demos: six canonical anchors — two CPU-sandbox references (`demo_cpu_single`/MBPP, `demo_cpu_math`), two GPU-sandbox demos (`demo_gpu_single`/`demo_gpu_shared`, KernelBench CUDA), and two resilience demos (`demo_cpu_quiesce` heartbeat keep-alive, `demo_gpu_snapshot` app-level snapshot/resume) — plus an **eight-way per-task CPU fan-out**, and two more that scale the **harness** (`demo_cpu_harness_scale`) and **mix task families into one stream** (`demo_cpu_mixtask`).

6 Demos — the demo set at a glance

The project ships a set of runnable demos in `code/demos/`, all built to one design (`docs/demos-program.md`): the policy is always **Qwen3-4B** trained by **slime**, the **harness is in-process**, and **AgentCore is the sandbox** — so

what varies from demo to demo is only the *sandbox substrate* (a CPU AgentCore Runtime / Code-Interpreter microVM vs a GPU g6 Runtime instance) and the *task*. The set has two layers. **Six canonical demos** anchor it: two CPU-sandbox references (`demo_cpu_single`, `demo_cpu_math`), two GPU-sandbox demos (`demo_gpu_single`, `demo_gpu_shared`), and two offline **resilience** demos (`demo_cpu_quiesce`, `demo_gpu_snapshot`). On top of them a generator (`gen_task_demos.py`) fans `demo_cpu_single` into **eight per-task CPU demos** (`demo_cpu_<task>`), each a self-contained recipe wired to one real Hugging Face dataset (covered in §"The per-task CPU fan-out" below). Two further demos sit outside both layers. `demo_cpu_harness_scale` fixes the task and sandbox and instead scales the **harness** itself, making the §"Scaling the harness too" law `throughput = min(inference, sandbox, harness)` concrete. `demo_cpu_mixtask` does the opposite of the fan-out: instead of one family per run it **interleaves several families into one training stream**, so every GRPO step draws a *multi-family* batch (both have their own subsection below). That makes **sixteen demos in all**. Each entry names the frontier task family from §"Tasks frontier labs run in a sandbox" it exercises and a real-world scenario it stands in for; the *Run status* column states honestly what has been **trained on hardware** versus what is a ready-to-run recipe.

The table below lists **all sixteen** demos, one brief row each, grouped by the three layers described above (six canonical, then the eight per-task CPU fan-out, then the two that scale a new dimension). Section 7 then deep-dives every one of them.

Demo	Substrate	Task	Frontier family (§3)	Run status
— canonical (6) —				
`demo_cpu_single`	CPU microVM	MBPP code repair	code generation [8]	**trained** — 10-step run + ablation sweep
`demo_cpu_math`	CPU microVM	math word problems	math reasoning [2]	smoke only (NR=3)
`demo_gpu_single`	GPU instance (g6)	KernelBench CUDA	GPU kernels [7]	**trained** — 10-step run

Demo	Substrate	Task	Frontier family (§3)	Run status
`demo_gpu_shared`	GPU instance (shared g6, Ray farm)	KernelBench CUDA	GPU kernels [7]	run attempted, failed (`rc=1`)
`demo_cpu_quiesce`	CPU microVM	heartbeat / quiesce	resilience (long-horizon)	offline, seconds
`demo_gpu_snapshot`	GPU instance	snapshot / resume	resilience (long-horizon)	offline, seconds
— per-task CPU fan-out (8) —				
`demo_cpu_gsm8k`	CPU microVM	GSM8K	math reasoning [2]	smoke (NR=3), eval 0.91
`demo_cpu_math500`	CPU microVM	MATH-500	math reasoning [2]	smoke (NR=3), eval 0.83
`demo_cpu_deepscaler`	CPU microVM	DeepScaleR	math reasoning [2]	smoke (NR=3), eval 0.70
`demo_cpu_humaneval`	CPU microVM	HumanEval	code generation [8]	smoke (NR=3), eval 0.81
`demo_cpu_gretelsql`	CPU microVM	Gretel text-to-SQL	SQL / data analysis	smoke (NR=3), eval 0.65
`demo_cpu_hotpotqa`	CPU microVM	HotpotQA (multi-hop)	RAG & search	smoke (NR=3), eval 0.21
`demo_cpu_xlam`	CPU microVM	xLAM-60k	tool use	smoke (NR=3), eval 1.00

Demo	Substrate	Task	Frontier family (§3)	Run status
`demo_cpu_mcp`	CPU microVM	ToolACE (MCP)	tool use	smoke (NR=3), `eval/mcp` 1.00
— scale a new dimension (2) —				
`demo_cpu_harness_scale`	CPU microVM	DeepScaleR math (fixed)	infra — scale the harness (§2)	offline sweep; no training run
`demo_cpu_mixtask`	CPU microVM	mixed: math + SQL + CTF + tool-use	cross-family regime (§2)	recipe only; no committed run

Only `demo_cpu_single` and `demo_gpu_single` carry committed training curves; `demo_cpu_math` has hardware *smoke* logs (not convergence); `demo_gpu_shared`'s remote-g6 run never passed its health check; the two resilience demos train no policy; the eight per-task fan-out demos each now carry a short `NR=3` smoke run (`rerun.log`) against the real CPU sandbox — enough to verify the loop, the checker, and a baseline held-out solve rate, but well short of convergence; `demo_cpu_harness_scale` has a *measured offline* throughput sweep but no committed training run; and `demo_cpu_mixtask` is the newest — its mixed-dataset builder is in place but the dataset is not yet built and it carries **no committed run at all** (a ready-to-run recipe only). This is stated plainly here so the measured results in §7 are read for what they are.

6.1 `demo_cpu_single` — MBPP code repair on a CPU microVM

Trains Qwen3-4B to fix Python until hidden unit tests pass, in a write → run → read → fix loop; each file/exec op runs inside one CPU AgentCore sandbox reached over `invoke_agent_runtime`, and the reward is 1.0 iff the MBPP asserts pass. Its dataset is MBPP (500 train / 8 held-out eval); the same task-agnostic harness also ships HumanEval, SQL, RAG, and tool-use seed sets in the folder. *Exercises (§3):* **code generation & competitive programming** (MBPP [8]) — few turns, cheap

checker, generation-bound. *Scenario:* a unit-test-driven coding assistant that iterates against a test runner.

6.2 `demo_cpu_math` — math word problems on a CPU microVM

The same in-process loop with `FAMILY=math`: the policy writes `solution.py`, and a trivial numeric checker scores it. Its datasets are real Hugging Face slices wired through `real_datasets.py` — GSM8K (200/40), MATH-500 (160/40), and DeepScaleR (400/40). *Exercises (§3):* **math & logical reasoning** (GSM8K/MATH-500 [2]) — typically single-turn, a long reasoning trace, strongly inference-bound. *Scenario:* reasoning-model math post-training, where the checker is a one-line assert and all the cost lives in generation.

6.3 `demo_gpu_single` — KernelBench CUDA on a GPU Runtime instance

The policy writes a CUDA `ModelNew`; a torch/CUDA checker rebuilds the reference on the GPU with matched seeds, asserts `allclose`, and reports a speedup. The checker runs **remotely** on a single GPU AgentCore Runtime backed by a g6 capacity-provider instance, so the trainer box keeps all eight of its GPUs for policy + inference. Its dataset is KernelBench (100 level-1 train / 20 level-2 eval). *Exercises (§3):* **GPU kernels / systems** (KernelBench [7]) — the one family whose *checker* itself needs CUDA; sandbox-bound. *Scenario:* a kernel-optimization agent whose reward requires running untrusted CUDA on a real GPU, isolated from training state.

6.4 `demo_gpu_shared` — KernelBench, many episodes packed onto one GPU

The same KernelBench task, but several concurrent episodes share **one** remote g6 GPU inside a single AgentCore instance Runtime (one shared `runtimeSessionId`, a per-episode `sub_session`); a Ray fractional-GPU farm inside the sandbox leases one `num_gpus=0.5` actor per episode (`floor(1/fraction)` warm-CUDA actors on one L4, since L4/L40S offer no MIG). The point is that the isolated approach — one whole g6 *per episode* — needs N instances at once, which **stalls on a capacity-starved fleet** (you cannot launch the instances, before cost even enters), whereas

the farm needs only one. *Exercises (§3):* **GPU kernels / systems** — the *scaling* answer for a sandbox-bound family when one-whole-GPU-per-episode is unavailable or too costly. *Scenario:* GPU-sandbox packing when a dedicated GPU per concurrent episode is either unobtainable on a capacity-starved fleet or too expensive.

6.5 `demo_cpu_quiesce` — microVM heartbeat across the weight-sync barrier

A standalone three-act demo: during slime's trainer → inference weight sync, generation stalls and idle sandbox sessions would be reclaimed on the ~15-minute idle timeout; a `SessionKeeper` heartbeat (with a quiesce gate) keeps them warm, and a recovery path re-seeds a session that dies anyway. Runs offline against an idle simulator. *Exercises (§3):* not a task — a resilience mechanism for **long-horizon families** (SWE, multi-agent) whose sessions must survive stalls. *Scenario:* keeping warm per-episode sandboxes alive across the seconds-long weight-sync pause without paying a cold start every step.

6.6 `demo_gpu_snapshot` — application-level snapshot / resume across a recycle

The sibling resilience demo: a GPU Runtime instance can be *recycled* and come back blank (AgentCore offers no VM/EBS snapshot), so accumulated workspace state is serialized to a store you own (S3 in production) and restored onto a fresh session. Run A (no snapshot) restarts from scratch; Run B resumes — printing the rework saved. Offline, ~1 s. *Exercises (§3):* resilience for **long-horizon GPU families** whose multi-hour sessions may outlive an instance. *Scenario:* surviving an instance recycle in a long RL run without discarding an episode's progress.

6.7 `demo_cpu_harness_scale` — scaling the harness tier

The one demo that varies **neither** the task nor the sandbox. It fixes both (DeepScaleR math, coding-shape, on a CPU microVM) and instead makes the **harness** the scalable dimension, to demonstrate the §"Scaling the harness too" law: rollout throughput is `min(inference, sandbox, harness)`, so once a deep async buffer and scaled inference/sandbox are in place, a single-process harness becomes the

ceiling. The scaling unit is a pool of **`W` warm in-process harness workers** (`HARNESS_WORKERS`, default 4) fed by one shared `asyncio.Queue`; the three tiers load-balance *differently*, which is the point — inference is stateless per **request** (slime's cache-aware SGLang router across E engines), the sandbox is sticky per **episode** (round-robin a fresh session across K Runtime ARNs), and the harness balances by **pull** (whichever warm worker is free takes the next episode — work-stealing, robust to wide per-episode latency). It ships two ways to see it: a fully **offline** scaling-law sweep (`harness_scaling_demo.py` — real `run_episode` on `LocalSandbox`, no AWS, runs in seconds) and the real slime integration behind the generate hook (`harness_pool.py` / `harness_pool_generate.py`, the warm-worker pool driving the `MiniClaudeHarness`). *Exercises (§3):* not a new task — the **infrastructure** lever of §2, using math reasoning [2] as the fixed workload. *Scenario:* a rollout fleet where you have already scaled inference and the sandbox and need to lift the third, easily-forgotten dial — the driver — without making it ad-hoc.

6.8 The per-task CPU fan-out — one recipe per real dataset

`demo_cpu_single` is a *task-agnostic* recipe: the same in-process harness and CPU sandbox train any family by setting `FAMILY` and pointing at a dataset. To make each real dataset a first-class, self-contained demo, `code/demos/gen_task_demos.py` **generates eight `demo_cpu_<task>` folders** from `demo_cpu_single` — each with its own vendored `agentcore_slime_demo/` library, `deploy/` Dockerfile, `data/` (that task's train/eval JSONL), a `run.sh` with `FAMILY` baked in, and a `verify_data.py` that proves the checker offline (the reference solution passes, a wrong one fails). Re-running the generator re-syncs all eight after a library change. They exercise the math, coding, SQL, RAG, and tool-use families of §3 on exactly the CPU-sandbox path of `demo_cpu_single`; the reward is RLVR (a programmatic checker), no LLM judge.

Each demo below is `demo_cpu_<name>`; the *Dataset* column gives a short label, with the full Hugging Face repo ID in the note that follows.

Demo (`demo_cpu_*`)	Family (§3)	Dataset	Reward	Train / eval
`gsm8k`	math	GSM8K	exact integer match	200 / 40
`math500`	math	MATH-500	numeric match (tol 1e-4)	160 / 40

Demo (`demo_cpu_*`)	Family (§3)	Dataset	Reward	Train / eval
`deepscaler`	math	DeepScaleR	numeric match (tol 1e-4)	400 / 40
`humaneval`	coding	HumanEval	the row's own `check()` unit test	140 / 24
`gretelsql`	sql	Gretel text-to-SQL	result-set equality (`sqlite3`)	300 / 40
`hotpotqa`	rag	HotpotQA	SQuAD-normalized exact match	300 / 40
`xlam`	tool use	xLAM-60k	arg-normalized gold-call match	300 / 40
`mcp`	tool use	ToolACE (MCP)	arg-normalized gold-call match	300 / 40

Full repo IDs: `openai/gsm8k`, `HuggingFaceH4/MATH-500`, `agentica-org/DeepScaleR-Preview-Dataset`, `openai/openai_humaneval`, `gretelai/synthetic_text_to_sql`, `hotpotqa/hotpot_qa` (distractor; 10-passage corpus), `minpeter/xlam-function-calling-60k-parsed`, `Team-ACE/ToolACE` (MCP-style schemas). Rewards are RLVR (arg-normalized matches are order-insensitive).

These eight are **fully scaffolded, data-prepared, and each now carries a short `NR=3` smoke run** (`rerun.log`) against the real CPU AgentCore sandbox — enough to verify the loop and checker end-to-end and to record a *baseline* held-out solve rate through the harness (from the two-rollout smoke: `eval/xlam` 1.00, `eval/gsm8k` 0.91, `eval/math500` 0.83, `eval/humaneval` 0.81, `eval/deepscaler` 0.70, `eval/gretelsql` 0.65, `eval/hotpotqa` 0.21, `eval/mcp` 1.00). These are **smoke baselines, not convergence** — a two-rollout run reflects the base policy's solve rate more than any learning — so the eight remain honest *ready-to-run recipes*, one per real dataset, whose convergence is still open work (§"Experiment plan"). Because each is an architectural clone of `demo_cpu_single`, the deep-dive in §7 covers them by covering `demo_cpu_single`; only `FAMILY` and the dataset differ.

6.9 `demo_cpu_mixtask` — one interleaved stream across task families

The fan-out pins **one** family per run; `demo_cpu_mixtask` does the opposite. It builds a single `mixed_train.jsonl` / `mixed_eval.jsonl` that **interleaves several families** (default **math + SQL + CTF + tool-use**, all pure-CPU) so that every GRPO rollout step draws a batch of prompts spanning multiple tasks — the setting a generalist policy is actually trained in, rather than a sequence of single-task runs. It is a CPU-microVM demo on the same skeleton as `demo_cpu_single`; only the *data composition* changes.

Why nothing in the harness or sandbox has to change. The mix works because each row is already **self-describing**: it carries `metadata.task = {family, task_id, prompt, spec, ⋯}`, and `generate._task_from_sample` reads `metadata.task.family` and rebuilds the right `EpisodicTask` subclass **per prompt** — so `run_episode` already dispatches the prompt, seed files, hidden checker, and reward *per row* (§"One harness, many task families"). The sandbox is likewise **task-agnostic** (`write_file` / `exec` / `read_file`): a math episode, a SQL episode, and a CTF episode are just different files written into three identical, isolated sandbox sessions. A step being multi-family is therefore transparent to both.

So mixtask needs no extra Runtimes — just more sessions. Because a Runtime is bound to one *image* and the sandbox is task-agnostic, the number of Runtimes tracks **distinct sandbox images, not distinct tasks**. The default mix here (math + SQL + CTF + tool-use) is **all pure-CPU**, so one generic CPU image serves every family: a mixed-family run is **one Runtime, many sessions** (one `runtimeSessionId` per episode) — exactly like a single-task run. You would add a *second* Runtime only if a family in the mix needed a **different image or substrate** — e.g. adding KernelBench (CUDA) pulls in the GPU Instances image, so you would register one ARN per image and route by `family` (the capability-routed dispatch of §"One image per Runtime, but a different env per row"). Task diversity alone never multiplies Runtimes; only image diversity does.

Why the reward scales do not clash. GRPO normalizes advantage **within a group** of `n_samples_per_prompt` repeats of the *same* prompt — hence the same family. Mixing happens **across groups within a step**, so the different families' reward scales never need reconciling: within-group normalization handles it, and only the prompt *sampler* mixes. The builder (`build_mixed_datasets.py`) offers a **balanced round-robin interleave** (every step-window spans the families evenly)

or a plain proportional shuffle, holds out a per-family eval slice *and* writes an interleaved `mixed_eval.jsonl`, and replicates the pool so one epoch spans the whole run (keeping save-only-at-end intact). Note the seeds here are the **hand-authored, templated `task_families.py` families**, not the real Hugging Face datasets of the fan-out — so the mix demonstrates the multi-family *training mechanics*, not dataset-scale coverage.

Exercises (§3): not a new family — the **cross-family training regime** itself, the closest of the demos to how a generalist policy is post-trained (§"Where agentic RL is used"). *Scenario:* training one policy across a diverse task portfolio in a single run, where each step's batch spans capabilities. *Run status:* newest demo — builder in place, dataset not yet built, **no committed run** (§"Experiment plan" carries its open convergence work).

7 Demos in depth — architecture, pipeline, lifecycle, and measured results

This section is the deep dive. Because the four canonical training demos share one architecture — in-process harness, AgentCore sandbox, slime trainer — the first subsection establishes the **shared skeleton** once (the pipeline, the step-by-step, the lifecycle, the scaling method, and the metric glossary); the second covers how to **size the two config knobs** every demo exposes; and the per-demo subsections then give only what differs — task, dataset, runtime, reward — and the **measured** results. The eight per-task CPU demos of §6 are architectural clones of `demo_cpu_single` (same skeleton, differing only in `FAMILY` and dataset), so rather than re-analyze each in full they are covered **together** in one consolidated subsection ("The eight per-task CPU fan-out demos in depth") that gives their measured smoke baselines and their shared bottleneck/plan; `demo_cpu_mixtask` is the same skeleton too but differs in a dimension worth its own short subsection — its dataset *interleaves* families rather than pinning one — so it gets one ("`demo_cpu_mixtask` in depth"). The Ray-farm, the two resilience demos, and the harness-scaling demo have a genuinely different topology and each carries its own diagram. Between them the per-demo subsections below now touch **all sixteen** demos.

A note on grounding: the numbers below are read from this project's own committed run artifacts (`train.csv`, the ablation CSVs under `demo_cpu_single/logs/`, and the `[sbx-op]`/`[sglang-gen]` records in the smoke and GPU logs). They come from

short runs (≈10 training steps, or smoke runs), so treat them as *shape and order-of-magnitude*, not as converged science. Where a metric is only *instrumented* — emitted by the code but not yet populated in a committed run — this is said explicitly rather than filled with an invented value.

7.1 The shared skeleton — one pipeline, one set of metrics

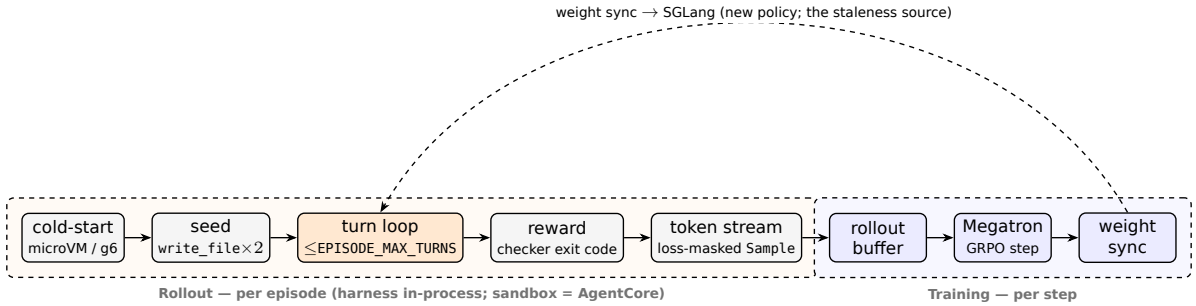


Figure 7: The demo pipeline, shared by all four training demos. An episode cold-starts its sandbox session, seeds the workspace, then runs the **turn loop** — each turn a generation through the proxy (`[sglang-gen]`) followed by a checker exec in the sandbox (`[sbx-op]`). The checker's exit code is the reward; `build_token_stream` emits a loss-masked `Sample` into the rollout buffer; Megatron takes a GRPO step and **weight-syncs** the new policy back to SGLang — the point at which rollouts generated on an older policy version become *stale*. The two labeled latencies are the two things the scaling decision (§"Scaling the sandbox") is made from.

Architecture. Every training demo is the topology of §"The call path": slime's `generate()` hook runs the agent loop in-process, each **model turn** goes through the proxy (which returns the exact sampled tokens + logprobs — the TITO property), and each **tool call** becomes one `invoke_agent_runtime` to an AgentCore sandbox where the checker produces the reward. The four training demos differ only in the sandbox ARN (CPU microVM vs GPU instance) and the task.

The four configs — trainer, inference, harness, sandbox. The training demos fix three of the four components and vary only the sandbox:

Component	Where it runs	Config (shared across the training demos)
Trainer	4 GPUs on the 8×L40S box	Megatron-LM, GRPO; `global_batch_size=32`; async checkpoint save; `ACTOR_GPUS=4`
Inference	4 GPUs on the same box	SGLang, 2 engines (2 GPUs each), `sglang-mem-fraction-static=0.7`, one **cache-aware router**; `ROLLOUT_GPUS=4`
Harness	in-process (CPU) next to the trainer	`run_episode`; fully-async `MODE=B` (deep buffer); `CONC=4` rollout concurrency; `INFER_CONC=0`; `EPISODE_MAX_TURNS=3`
Sandbox	**the only variable** — remote AgentCore	CPU microVM (arm64) or GPU `g6` instance (amd64); one `runtimeSessionId` per episode; the `SANDBOX` env var selects it

Each per-demo subsection states that demo's **sandbox** row; the trainer, inference, and harness rows are identical unless noted. This is the disaggregation of §"Agentic RL and its reference architecture" made concrete: GPUs go to training and inference, the harness rides cheap CPU in-process, and the sandbox is the isolated remote seam.

Step by step (one episode). Cold-start the session's microVM (or launch the g6) → seed `PROBLEM.md` + `run_checks.py` → the turn loop (≤ `EPISODE_MAX_TURNS`): generate via the proxy (`[sglang-gen]`), write the artifact, exec the checker (`[sbx-op]`), feed any failure back → reward from the checker's exit code → `build_token_stream` → a loss-masked `Sample` → the rollout buffer → a Megatron GRPO step → weight sync back to SGLang.

Lifecycle. One session = one episode: the first invoke cold-starts the microVM (or launches the g6), the same `runtimeSessionId` re-routes to the warm VM, and the session is reclaimed on idle (~15 min CPU / 600 s GPU) or on max lifetime (8 h microVM / up to 14 d instance). Under asynchronous training, the pause during weight sync can trip the idle clock — which is what the resilience demos address.

Scaling method. Following §"Scaling the sandbox": read `perf/sandbox_ms_*` against `perf/sglang_gen_ms_*`, then grow the dominant side — inference (more SGLang replicas behind the **cache-aware** router) when generation dominates, or the sandbox (more microVM sessions, or a pool of Runtime ARNs round-robined **sticky per episode**; on GPU, the Ray farm) when the checker dominates. The demos are chosen so both cases actually occur: MBPP/math are generation-bound, KernelBench is sandbox-bound.

What is measured — and what is only instrumented. The metric glossary the deep-dives use:

Metric	Meaning	Status in these demos
`rollout/raw_reward`, `eval/<task>`	mean binary checker reward per step; held-out solve rate	**measured** (per step + eval)
`perf/actor_train_tok_per_s`, `perf/actor_train_tflops`	trainer throughput and achieved FLOPs	**measured**
`perf/mfu`	model FLOPs utilization = TFLOPS ÷ 362 (L40S peak)	**measured** (ablation CSVs)
`perf/wait_time_ratio`	fraction of the step the trainer waits on rollout — the pipeline-balance / GPU-idle proxy	**measured**
`perf/goodput_tok_per_s`	throughput × (1 − discard ratio)	**measured**, but = throughput (discard = 0)
`train/pg_clipfrac`	PPO/GRPO clip fraction	**measured**, uniformly 0.0
`[sglang-gen]`, `[sbx-op]` p50/p95	**per-turn inference latency** (generation) and per-op sandbox latency	**measured** in the run logs

Metric	Meaning	Status in these demos
`perf/step_time`	per-step wall-clock — **batch training** + rollout	**measured** (≈ 23–78 s/step, `demo_cpu_single`; 110–804 s/step on `demo_gpu_single`, where the remote-g6 checker dominates)
weight-sync latency	broadcasting new weights trainer → SGLang each step	**not separately recorded** here (folded into `step_time`); ≈ 20–30 s for a 1T model at frontier scale [29]
staleness (`v_use` − `v_gen`), discard/staleness ratio, importance-sampling ratio (ρ, TIS), `num_turns`, buffer depth, samples/s, direct GPU utilization	off-policy and horizon signals	**instrumented or configured only** — no value recorded yet

The last row matters: the code emits `[sbx-op]`/`[sglang-gen]` and the derived columns exist, but staleness, discard ratio, and the importance-sampling ratio are described in `docs/observability.md` as slime keys we would read — they are not populated in any committed run here, so no number is claimed for them below. Trainer-vs-inference GPU utilization is likewise not directly sampled; the split is *configured* (4 trainer + 4 inference GPUs), and `perf/wait_time_ratio` is the recorded proxy.

7.2 Sizing the config — response length per turn, and turns per episode

Every demo exposes two knobs that follow directly from the episode shapes of §“Tasks frontier labs run in a sandbox”. Getting them right is what makes a run efficient rather than wasteful or broken.

Max response length per turn (`--rollout-max-response-len`). This is the per-turn generation budget. Set it from the **p95 of the policy's own per-turn response length** for your family (from §3: math reasoning traces run 1k–8k and up to 32k tokens; a raw code submission is 100–800 but a reasoning model prepends thousands; a CUDA kernel is 200–1,000; a tool call is 50–300). Size to that p95 with headroom. **Too small** truncates the reasoning trace mid-thought: the checker then scores an incomplete artifact (depressing reward), and the truncated sequence is still trained on (a biased gradient). **Too large** wastes KV cache and slows decode, because SGLang reserves capacity per sequence. In these demos `enable_thinking=False` keeps Qwen3-4B's per-turn responses short, so a modest budget suffices; a reasoning policy on hard math would need a far larger one — measure before committing.

Max turns per episode (`EPISODE_MAX_TURNS`, default 3). This caps how many generate→exec rounds an episode may take. It is the single biggest lever on episode cost: each extra turn adds one full generation *and* one sandbox round-trip, and it grows the context every subsequent turn must attend over — so per-episode cost ≈ `turns × (generation + exec + observation)`, and the trainer's ref/actor forward passes scale with the resulting sequence length. More turns raise the *ceiling* on reward (more chances to read a failure and fix it) but lengthen episodes, which under asynchronous training means **more staleness** (a long episode spans more weight-sync barriers) and more sandbox operations. Set it from the family's horizon: single-turn families (math, instruction following) → 1; iterative code/kernel/SQL → 3–5; long-horizon SWE / web / multi-agent → 15–50+, at which point a **warm, long-lived session** (§`demo_cpu_quiesce`) becomes necessary. It is controlled per run as an env var in `run.sh`, threaded into the harness as `max_turns` — no redeploy needed. These demos use `EPISODE_MAX_TURNS=3` and rollout concurrency `CONC=4`.

7.3 Experiment plan — training each demo to convergence

A demo is "done" only when it shows **learning to convergence**, not a smoke run (`docs/demos-program.md`). What is committed today is deliberately short: `demo_cpu_single` and `demo_gpu_single` have ~10-step runs (enough to read the bottleneck, not to converge), `demo_cpu_math` and each of the eight per-task fan-out demos have `NR=3` smoke logs (a two-rollout end-to-end sanity check with a baseline held-out solve rate, not a learning curve), and `demo_gpu_shared`'s run failed. The

plan to convergence has the same shape for every training demo and differs only in the step budget and which side gets scaled once the bottleneck is measured.

The convergence bar (all training demos). (1) Train enough steps that the **held-out eval solve rate rises then plateaus** — log `eval/<task>` every `EVAL_INTERVAL` steps, not the default ~10-step run. (2) `rollout/raw_reward` and `eval/<task>` trend up then flat on the folder's `train.png`. (3) `train/grad_norm` and `train/kl_loss` stay sane (the policy is moving, not diverging). (4) Only then apply the §"Scaling the sandbox" decision: scale the *measured* bottleneck and show the `perf/step_time` improvement — confirming the other side would not have helped.

Because the metric that decides scaling (`perf/sandbox_ms_*` vs `perf/sglang_gen_ms_*`) is measured *per demo*, the experiment plan is per demo too — the bottleneck column below is what each demo's own latencies (in its subsection) already show:

Demo	Eval metric	Measured bottleneck	Scale	Convergence note
`demo_cpu_single`	`mbpp`	generation (sandbox 0.12 s ≪ gen 1.76 s)	inference — SGLang replicas	run past 10 steps until `eval/mbpp` plateaus
`demo_cpu_math`	`math`	generation (trivial checker, long traces)	inference	long-reasoning → largest `--rollout-max-response-len`
`demo_gpu_single`	`kernelbench`	sandbox (exec 8.5 s ≫ gen 2.5 s; high `wait_time_ratio`)	sandbox — more g6 or the farm	budget the ~90 s torch + g6 cold start
`demo_gpu_shared`	`kernelbench`	sandbox, packed	Ray fractional actors per GPU	fix the `rc=1` queued run first, then converge

That table says *what* to scale; the protocol below says *how far to run* each

demo to earn the right to scale — a concrete step budget (NR), eval cadence (EVAL_INTERVAL), a target held-out plateau, and a wall-clock estimate. All share the committed config (global_batch_size=32, CONC=4, EPISODE_MAX_TURNS=3):

Demo	Steps (NR)	Eval ev-ery	Held-out start → target plateau	Est. wall-clock*
demo_cpu_single	150–200	10	eval/mbpp 0.56 → ≥ 0.80	~1–4 h
demo_cpu_math	150–200 /split	10	eval/gsm8k 0.94 → flat; MATH-500 / DeepScaleR climb from cold	~2–6 h /split
demo_gpu_single	100–150	10	eval/kernelbench 0.65 → ≥ 0.80	~4–24 h
demo_gpu_shared	100–150	10	same task as the single-GPU demo, on **one** g6	≈ single-GPU/step

* *Wall-clock is each demo's measured perf/step_time (§ metric glossary) times the step budget — a planning extrapolation, not a measured convergence time. The step budgets (chosen so MBPP's 500-row / KernelBench's 100-row splits see ~10 epochs at GBS=32) and the target plateaus are likewise **targets**: no demo has yet been run to plateau — the committed runs are the ~10-step demo_cpu_single / demo_gpu_single, the NR=3 demo_cpu_math smoke, and the failed demo_gpu_shared. The GPU range is wide because perf/step_time spans 110–804 s: the remote-g6 cold start (~90 s torch install + ~123 s first ping) and per-episode checker variance dominate it.*

The **eight per-task CPU demos** (§"The per-task CPU fan-out") inherit demo_cpu_single's plan unchanged — they are generation-bound (cheap checker, math/coding/sql/rag/tool use), so each converges by running past the smoke budget while tracking its own eval/<task> every 10 steps and, if generation dominates, adding SGLang replicas. Concretely: NR ≈ 100–200 (scaled to each 140–400-row split, so ~10 epochs at GBS=32), EVAL_INTERVAL=10, ~1–3 h each on CPU — the same 23–78 s/step envelope as demo_cpu_single. Running these eight to convergence is the fan-out's open work item.

7.4 `demo_cpu_single` in depth — MBPP on a CPU microVM

- **Task / harness:** MBPP multi-turn Python repair; task-agnostic `run_episode`, in-process.
- **Dataset:** MBPP 500 train / 8 eval (`build_dataset.py`), plus family seed sets in `data/`.
- **Runtime:** one CPU AgentCore Runtime (`sandbox_server_agent.py`, arm64, `deploy/Dockerfile.sandbox.cpu`); `SANDBOX=agentcore-runtime` (or `agentcore` for the Code-Interpreter microVM, `local` for offline CI). All 8 L40S serve the policy (4 train + 4 inference); 0 env GPUs.
- **Reward:** run the candidate against hidden MBPP asserts; 1.0 iff exit code 0.
- **Sandbox scaling:** round-robin a fleet of sandbox Runtime ARNs (`AGENTCORE_SANDBOX_RUNTIME_ARNS`, `deploy_cpu_sandbox_fleet.py`, `run_runtime_ablation.s` sticky per episode.

Measured results. Over the committed 10-step run (`train.csv`), `rollout/raw_reward` moved in the ~0.5–0.94 band (0.875 at step 1, dipping to 0.5 at step 9 — it never reaches 1.0) and held-out `eval/mbpp = 0.5625` at step 9. Trainer throughput `perf/actor_train_tok_per_s ≈ 1.5k–3.3k` tok/s, `perf/actor_train_tflops ≈ 8.9–20.4`, `perf/step_time ≈ 23–78 s`, `perf/wait_time_ratio ≈ 0.39–0.84`, and `train/pg_clipfrac = 0.0` at every step. The runtime-count ablation (`logs/ablation_summary.csv`, sweeping 1/2/4 sandbox Runtimes) adds `mean_mfu ≈ 0.018–0.043` (2–4 % of the L40S 362-TFLOPS peak), `mean_goodput_tok_per_s ≈ 1.1k–2.6k` (equal to throughput — discard ratio 0), and a wider `wait_time_ratio` spread. Real latencies (from `smoke_realaws.log`): **sandbox** ops p50 123 ms / p95 900 ms (`write_file` p50 94 ms, exec p50 123 ms, `ping` p50 546 ms); **SGLang** decode p50 1.76 s / p95 5.0 s. Reading them: generation (1.76 s) ≫ sandbox (0.12 s), so this family is **generation-bound** — scale inference, exactly as §3 predicts for a cheap-checker code task. **Plan to convergence:** run `NR ≈ 150–200` steps (MBPP is 500 rows ≈ 16 steps/epoch at `GBS=32`, so ~10 epochs) with `EVAL_INTERVAL=10`, until `eval/mbpp` climbs from its 0.56 start to a ≥ 0.80 plateau — an estimated ~1–4 h at the measured 23–78 s/step; then add SGLang replicas (the generation-bound side) and confirm `perf/step_time` drops while `eval/mbpp` holds.

7.5 `demo_cpu_math` in depth — math word problems on a CPU microVM

- **Task / harness:** `FAMILY=math`; write `solution.py`, numeric checker; same in-process loop.
- **Dataset:** real HF slices via `real_datasets.py` — GSM8K 200/40, MATH-500 160/40, DeepScaleR 400/40.
- **Runtime / reward / scaling:** identical to `demo_cpu_single` (CPU microVM; 1.0/0.0 on the checker's exit code; scale inference).

Measured results — smoke only. The committed artifacts are hardware *smoke* runs (`NR=3`), not convergence runs: on `smoke_gsm8k.log` `rollout/raw_reward` was 0.9375 then 1.0 across the two training rollouts, with held-out `eval/gsm8k = 0.9375`; a matching `smoke_deepscaler.log` exists. Both carry real `[sbx-op]`/`[sglang-gen]` records. Treat these as a *sanity signal that the loop and the numeric checker work*, not as a learning curve — the README states the convergence bar is deliberately not met by a smoke run. The family is nonetheless clearly **inference-bound**: the reasoning traces dominate wall-clock while the numeric checker is milliseconds, so the `[sglang-gen]` records — not the `[sbx-op]` records — set the step time, and the scaling answer is more SGLang capacity behind the cache-aware router. **Plan to convergence:** run `NR` $\approx$ `150–200` per split with `EVAL_INTERVAL=10`, tracking `eval/math` on GSM8K / MATH-500 / DeepScaleR (the per-task fan-out demos `demo_cpu_gsm8k`/`demo_cpu_math500`/`demo_cpu_deepscaler` package exactly these) — `eval/gsm8k` starts near its 0.94 smoke value and should flatten quickly, while MATH-500 and DeepScaleR climb from cold; size `--rollout-max-response-len` to the reasoning-trace p95 and scale SGLang, since the bottleneck is generation. Estimate ~2–6 h/split — longer than `demo_cpu_single` because the reasoning traces are longer.

7.6 `demo_gpu_single` in depth — KernelBench on a GPU Runtime instance

- **Task / harness:** write a CUDA `ModelNew`; the harness is in-process, only the checker is remote.
- **Dataset:** KernelBench 100 level-1 train / 20 level-2 eval (`build_dataset.py --dataset kernelbench`).

- **Runtime:** one GPU AgentCore Runtime on the `acs_gpu_cp` capacity provider (a real `g6.xlarge` / L4, amd64, `deploy/Dockerfile.sandbox.gpu`, ECR `acs-sandbox-gpu`); `SANDBOX=agentcore-runtime`. **torch 2.5.1 is pip-installed at container start (~90 s cold)** to dodge the 2048 MB image cap. The trainer box's 8 GPUs all go to slime (4 train + 4 inference); the checker's GPU is the remote g6.
- **Reward:** rebuild the reference on CUDA with matched seeds, assert `allclose`, report speedup; failure text feeds the next turn.
- **Sandbox scaling:** more g6 instances (isolated) or the Ray farm of `demo_gpu_shared` (packed).

Measured results. Over the committed 10-step run, `rollout/raw_reward` moved in the ~0.28–0.94 band (0.81 at step 1) and `eval/kernelbench = 0.65` (CSV) / 0.725 (log). Trainer throughput `perf/actor_train_tok_per_s` ≈ 2.9k–4.0k tok/s, `perf/actor_train_tflops` ≈ 19–26, `perf/step_time` ≈ 110–804 s. `perf/wait_time_ratio` ≈ 0.79–0.96 — high, meaning the trainer spends most of the step waiting on rollout + the remote GPU checker. Real latencies (from the GPU run log): **sandbox** exec p50 8.5 s / p95 17.3 s / max 24 s; the first `ping` p50 ≈ 123 s (the g6 EC2 cold launch, amortized once per session); `write_file` p50 170 ms. **SGLang** decode p50 2.5 s / p95 6.4 s. Reading them: exec (8.5 s) ≫ generation (2.5 s), so KernelBench is **sandbox-bound** — scale the sandbox (more instances, or the Ray farm), exactly as §3 predicts. The high `wait_time_ratio` is the same story from the trainer's side. **Plan to convergence:** run `NR` ≈ 100–150 steps (KernelBench is 100 train rows) with `EVAL_INTERVAL=10`, until `eval/kernelbench` plateaus above its ~0.65–0.73 start toward ≥ 0.80; budget the one-time ~90 s torch install + ~123 s g6 cold launch per session, and expect ~4–24 h given the 110–804 s/step range; because it is sandbox-bound, scale the sandbox (more g6 instances or the Ray farm) rather than inference.

7.7 `demo_gpu_shared` in depth — packing many episodes onto one GPU

- **What differs from `demo_gpu_single`:** one line — `SANDBOX=agentcore-ray-farm`. The sandbox image bakes Ray (`deploy/Dockerfile.sandbox.gpu.shared`), deployed as `acs_sandbox_gpu_b4` via `redeploy_gpu_sandbox_shared.py`. Farm knobs (`B4_GPU_FRACTION`, `B4_MAX_SANDBOXES`, `B4_PER_SANDBOX_MEM_FRACTION`)

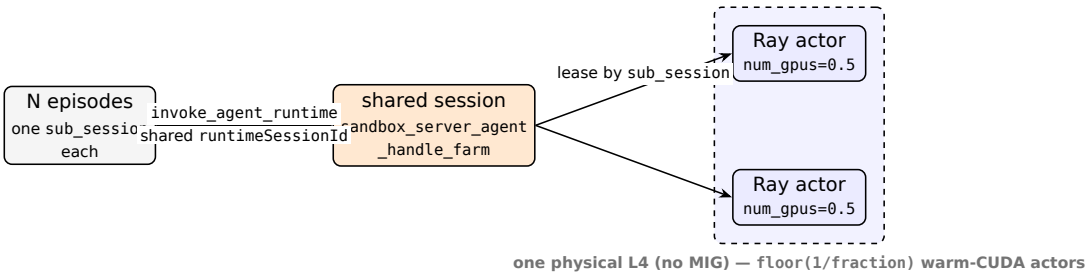


Figure 8: The Ray fractional-GPU farm of `demo_gpu_shared`. All concurrent episodes invoke *one* AgentCore GPU Runtime under a single shared `runtimeSessionId`; inside it, `_handle_farm` leases one fractional-GPU Ray actor per episode (keyed by a per-episode `sub_session`). With `num_gpus=0.5`, two warm-CUDA actors share the one L4; at `fraction=0.25`, four. Because L4/L40S offer no MIG, isolation is software-only (per-process memory caps + an `nvcc` `flock`, since a non-MIG L4 wedges under sustained compile storms).

are runtime env on the g6, retunable without a rebuild; `INVOKE_MAX_RETRIES` is raised for the minutes-long cold launch.

- **Task / dataset / reward:** identical to `demo_gpu_single` (KernelBench, `all-close`/speedup).

Results — honest status. The fractional-GPU *packing* has been validated on hardware (a non-MIG L4 wedges under sustained `nvcc` load, which `fraction=0.5` mitigates), but the committed end-to-end *training* run was queued and exited `rc=1` (`b_queue.out`, 2026-09-15); its queued log holds 0 completed reward steps. So there is no convergence curve for this demo yet — use `demo_gpu_single`'s per-episode numbers as the baseline this farm is designed to pack several-fold onto one GPU. This is the sandbox-side answer to KernelBench's sandbox-bound profile, and its **primary motivation is capacity, not just cost**: the isolated-per-session path needs *N whole g6 instances at once*, which stalls on a capacity-starved fleet — you cannot launch the instances at all (the committed run's own `RuntimeClientEr-ror`/health-check timeouts are that fragility showing through). Packing N episodes onto **one** g6 needs only a single instance, at the price of software-only isolation — so the trade is: isolated = N clean instances (N× cost, and unobtainable when the fleet is starved) versus shared = one instance, weaker isolation. **Plan to convergence:** first make the queued run pass (it exits `rc=1` today), then run the same `NR ≈ 100–150` / `EVAL_INTERVAL=10` protocol as `demo_gpu_single` — but on **one** g6 with two `num_gpus=0.5` actors instead of N instances — and compare goodput-per-GPU against `demo_gpu_single`'s one-instance-per-episode baseline to quantify the

packing win.

7.8 `demo_cpu_quiesce` and `demo_gpu_snapshot` in depth — surviving the pause and the recycle

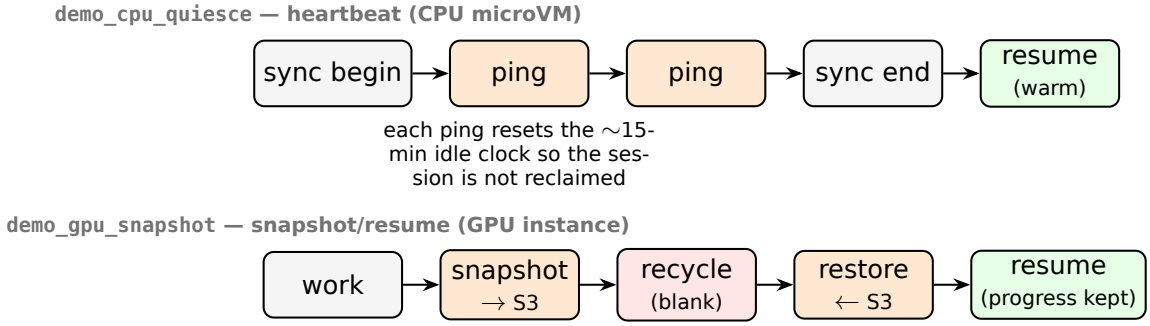


Figure 9: The two resilience levers. **Heartbeat** (top) keeps a *live* session warm across the seconds-long weight-sync pause by resetting the idle clock — no state is lost. **Application-level snapshot** (bottom) restores a *lost* session after a genuine instance recycle by serializing the workspace to S3 and restoring it onto a fresh session — because AgentCore offers no VM/EBS snapshot. Heartbeat prevents loss; snapshot recovers from it.

Neither demo trains a policy, so neither reports RL metrics; both run offline in seconds and demonstrate a mechanism the training demos would compose in production.

- **`demo_cpu_quiesce`** — the reusable `SessionKeeper` (a process-wide registry + one background heartbeat loop driven by the trainer's `sync_begin()`/`sync_end()` barrier) and `KeptSandbox` (wraps any `Sandbox`), in `session_keeper.py`. The three-act arc: ACT 1 naive "stop invoking" → the session is idle-reclaimed; ACT 2 the heartbeat keeps it warm across an arbitrarily long pause while the gate holds new invokes during sync; ACT 3 recover-by-reseed if a session dies anyway. The heartbeat lives **outside** the harness by design, so no harness grows its own keepalive.
- **`demo_gpu_snapshot`** — `snapshot_workspace()` / `restore_workspace()` in `snapshot_store.py`, backend-agnostic (`LocalSnapshotStore` offline, `S3SnapshotStore` for production — no mount, AZ-agnostic). The A/B arc: RUN A (no snapshot) redoes from step 0 after a recycle; RUN B snapshots before the recycle and restores onto a fresh session, printing the rework saved

(e.g. “6 vs 4 steps → 2 saved”). This is explicitly *not* a VM/EBS snapshot — see `docs/gpu-snapshot.md` for why an application-level snapshot is the only feasible option on AgentCore.

7.9 `demo_cpu_harness_scale` in depth — scaling the harness worker pool

This demo makes the §“Scaling the harness too” law operational. It holds the task and sandbox fixed and sweeps the harness’s own scaling unit — a pool of `W` warm in-process workers — to show that once inference and the sandbox are provisioned, rollout throughput is `min(inference, sandbox, harness)` and the driver is the third dial. Its topology differs from the training demos (a work-stealing worker pool over three independently-balanced tiers), so it carries its own diagram.

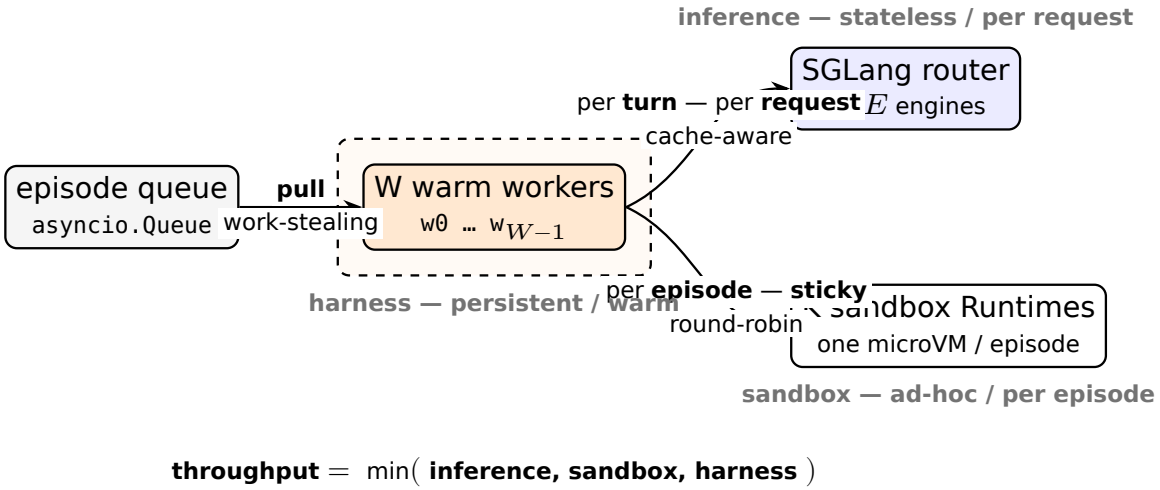


Figure 10: The three tiers of `demo_cpu_harness_scale`, and how each load-balances. **Harness**: `W` warm workers pull episodes off a shared queue — work-stealing balances by *availability*, robust to the wide, variable latency of multi-turn agentic episodes. **Inference**: each turn’s `/generate` is spread across the E SGLang engines by the cache-aware router — balanced per *request*, so one episode’s turns may land on different engines. **Sandbox**: a fresh microVM session per episode, round-robined across K Runtime ARNs and *sticky* for the episode’s life (it holds the episode’s files). Raising `W` lifts the harness tier until throughput hits `min(inference, sandbox)`.

- **Task / harness:** DeepScaleR math in coding-shape (write `solution.py`

setting a module-level `answer`; a numeric-assert checker scores it, RLVR, no LLM judge). The real integration drives the `MiniClaudeHarness` (`write_file`/`bash`/`read_file` over the Anthropic Messages API to slime's in-process adapter); the offline sweep drives the family-agnostic `run_episode`.

- **Dataset:** DeepScaleR 400 train / 40 eval (`data/deepscaler_{train,eval}.jsonl`), numeric-answer subset.
- **Two artifacts:** an **offline** scaling-law sweep (`harness_scaling_demo.py`: real `run_episode` on a `LocalSandbox`, inference and sandbox modeled as a concurrency ceiling + service time, `W` swept — no AWS/GPU, seconds) and the **real slime integration** (`harness_pool.py` warm-worker pool behind `--custom-generate-function-path agentcore_slime_demo.harness_pool_generate.gen` with `HARNESS_WORKERS=W` as the knob; requires `--sglang-tool-call-parser qwen25 --sglang-reasoning-parser qwen3`, the tool-parser fix without which the mini-harness tool loop scores a structural zero).

Measured results — offline sweep only. There is **no committed slime training run** for this demo; the numbers below are from the offline model (`harness_scaling_demo.py`) on the dev box — measured wall-clock throughput, with the per-tier ceilings computed from the service times observed during the run. With the inference ceiling set to `CINF=4` in-flight @ ~100 ms (≈ 40 ep/s) and the sandbox to `CSBX=8` @ ~50 ms + the real checker subprocess (≈ 80 ep/s), throughput rose ~linearly with `W` while the **harness** bound — 5.0, 9.6, 17.6, 29.5, 33.2 ep/s at `W` = 1, 2, 4, 6, 8 — then **plateaued at ~37 ep/s** once **inference** bound (raising `W` to 12 bought little more: 36.6 ep/s, and the committed sweep marks `W=8` and 12 as inference-bound). The `min()` law is visible directly: harness-bound at small `W`, then the ceiling relocates to inference, the next-scarcest tier — to push past ~40 ep/s you would have to lift `CINF` too, exactly the joint-scaling point of §2. Every episode solved (48/48), so the checker path is real, not mocked away. The sweep is an **unseeded offline simulation**: absolute ep/s are machine-dependent and vary run to run — treat them as an *illustration of the `min()` law*, not a slime rollout-throughput measurement.

Honest scope. The offline sweep models inference/sandbox as capacity + latency; it demonstrates the concurrency (IO-bound) ceiling that warm asyncio workers lift, not the CPU/GIL ceiling of a single process (tokenization, trajectory flattening) — passing that needs separate processes or dedicated harness Runtimes, the placement question §2 leaves open. The real training path is wired and documented but, like the fan-out demos, has no committed convergence run; running it is open work.

7.10 The eight per-task CPU fan-out demos in depth — one recipe per real dataset

The eight `demo_cpu_<task>` folders (§"The per-task CPU fan-out") are **architectural clones of `demo_cpu_single`** — the same in-process harness, the same CPU AgentCore microVM sandbox, the same slime trainer and shared skeleton (§"The shared skeleton"). Only two things differ per folder: **`FAMILY`** and the **real Hugging Face dataset**. So everything the `demo_cpu_single` deep-dive establishes — pipeline, the four configs, lifecycle, metric glossary — applies to each unchanged, and three properties are **shared by all eight**, stated here once so each subsection below need only give its own dataset, checker, and measured number:

- **What was run (shared).** Each is fully scaffolded and data-prepared, and each carries a short **`NR=3` smoke run** (`rerun.log`) against the *real* CPU sandbox — a two-rollout end-to-end pass that exercises the loop and checker and records a **baseline held-out solve rate**, well short of convergence. The `eval/<task>` numbers below therefore reflect the **base Qwen3-4B policy** through the harness, *not* learning: they are **smoke baselines, not curves.**
- **Bottleneck (shared).** All eight are **generation-bound** (§3, cheap-checker CPU families): the checker is milliseconds, the policy's response is the wall-clock, so `[sglang-gen]` — not `[sbx-op]` — sets step time. **Scale inference (SGLang replicas behind the cache-aware router), never sandboxes.**
- **Plan to convergence (shared).** Run **`NR` ≈ `100`–`200`** steps (scaled to each 140–400-row split, ~10 epochs at `GBS=32`), `EVAL_INTERVAL=10`, at the same **23–78 s/step** CPU envelope (~1–3 h each), tracking the demo's own `eval/<task>` from its smoke baseline toward a plateau. Running all eight to convergence is the fan-out's open work item.

7.10.1 `demo_cpu_gsm8k` in depth — grade-school math word problems

- **Task / dataset:** `FAMILY=math`; GSM8K (`openai/gsm8k`), 200 train / 40 eval. Write `solution.py`, one integer answer.
- **Checker / reward: exact integer match** against the gold answer; 1.0/0.0.
- **Measured (smoke):** `eval/gsm8k` **0.91** — the base policy already solves most of this easy split, so its curve should **flatten quickly**; it is the fan-out's sanity anchor.

7.10.2 `demo_cpu_math500` in depth — competition-style math

- **Task / dataset:** `FAMILY=math`; MATH-500 (`HuggingFaceH4/MATH-500`), 160 train / 40 eval.
- **Checker / reward: numeric match** (tolerance 1e-4) on the extracted answer.
- **Measured (smoke):** `eval/math500` **0.83** — harder than GSM8K, so more headroom to climb; the longer reasoning traces make its `--rollout-max-response-len` the one to size up first.

7.10.3 `demo_cpu_deepscaler` in depth — RL-scale math (largest split)

- **Task / dataset:** `FAMILY=math`; DeepScaleR (`agentica-org/DeepScaleR-Preview-Dataset`), 400 train / 40 eval — the **largest** fan-out split, so nearest to a real epoch budget.
- **Checker / reward: numeric match** (tolerance 1e-4).
- **Measured (smoke):** `eval/deepscaler` **0.70** — the lowest of the math three (hardest cold), the most room to show learning; also the workload `demo_cpu_harness_scale` reuses.

7.10.4 `demo_cpu_humaneval` in depth — function-synthesis coding

- **Task / dataset:** `FAMILY=coding`; HumanEval (`openai/openai_humaneval`), 140 train / 24 eval. Write a function body.
- **Checker / reward: the row's own `check()` unit test** run in the sandbox; 1.0 iff it passes.
- **Measured (smoke):** `eval/humaneval` **0.81**. Same shape as `demo_cpu_single` (MBPP) but with the canonical HumanEval tests — a real code-execution checker rather than a numeric assert.

7.10.5 `demo_cpu_gretelsql` in depth — text-to-SQL

- **Task / dataset:** `FAMILY=sql`; Gretel synthetic text-to-SQL (`gretelai/synthetic_text_to_sql`), 300 train / 40 eval. Emit one SQL query.
- **Checker / reward: result-set equality** — the query is executed against **`sqlite3`** (Python stdlib, so no extra image dep) and its rows compared to gold.

- **Measured (smoke):** `eval/gretelsql` **0.65**. Cheapest checker of all (an in-process SQLite query), textbook generation-bound.

7.10.6 `demo_cpu_hotpotqa` in depth — multi-hop RAG

- **Task / dataset:** `FAMILY=rag`; HotpotQA (`hotpotqa/hotpot_qa`, distractor; a 10-passage corpus per question), 300 train / 40 eval.
- **Checker / reward: SQuAD-normalized exact match** of the answer span.
- **Measured (smoke):** `eval/hotpotqa` **0.21** — the **hardest cold** of the eight (multi-hop over 10 passages), so the most interesting convergence target. **Note:** it is the one mild exception to "generation-bound" — the 10-passage observations grow the *context* each turn, but the checker is still trivial, so it is generation/**context**-bound, still not sandbox-bound.

7.10.7 `demo_cpu_xlam` in depth — function-calling / tool use

- **Task / dataset:** `FAMILY=tool use`; xLAM-60k (`minpeter/xlam-function-calling-60k-parsed`), 300 train / 40 eval. Emit the tool call(s) for a request.
- **Checker / reward: arg-normalized gold-call match** (order-insensitive over arguments), RLVR.
- **Measured (smoke):** `eval/xlam` **1.00** — the base policy already nails this smoke slice, so this demo's value is verifying the tool-call parsing/checker path end-to-end more than showing a climb.

7.10.8 `demo_cpu_mcp` in depth — MCP-style tool schemas

- **Task / dataset:** `FAMILY=tool use`; ToolACE (`Team-ACE/ToolACE`, MCP-style schemas), 300 train / 40 eval — the same family as `xlam` but against MCP-shaped tool definitions.
- **Checker / reward: arg-normalized gold-call match**, RLVR.
- **Measured (smoke):** the smoke run (`rerun.log`, `eval-interval 2`) committed **`eval/mcp` 1.00** on the two-rollout held-out set — a *baseline solve rate*, not convergence: with only two rollouts against a small tool-call gold set the base policy already matches, so read it as "the checker and MCP-style tool loop run end-to-end," not as a learned gain. The shared fan-out plan applies from here.

7.11 `demo_cpu_mixtask` in depth — one interleaved multi-family stream

Same skeleton as `demo_cpu_single` (in-process harness, CPU microVM sandbox, slime), but it differs in a dimension none of the others touch: the **data composition**. Instead of pinning one family per run, its builder (`build_mixed_datasets.py`) produces a single `mixed_train.jsonl` / `mixed_eval.jsonl` that **interleaves several families** (default **math + SQL + CTF + tool-use**, all pure-CPU) so every GRPO rollout step draws a **multi-family batch** — the setting a generalist policy is actually trained in.

- **Why no architectural change is needed.** Each row is **self-describing** (`metadata.task = {family, task_id, prompt, spec, ⋯}`), and `generate._task_from_sample` rebuilds the right `EpisodicTask` subclass **per prompt**, so `run_episode` already dispatches prompt, seed files, checker, and reward per row (§"One harness, many task families"). The sandbox is task-agnostic, so a math, a SQL, and a CTF episode are just different files in three identical, isolated microVMs. And crucially (§"`demo_cpu_mixtask`"), **one Runtime, many sessions** still holds — all four default families are pure-CPU and share the one generic image, so mixing multiplies *sessions*, not *Runtimes*.
- **Why the reward scales don't clash.** GRPO normalizes advantage **within a group** of `n_samples_per_prompt` repeats of the *same* prompt — hence the same family — so the different families' reward magnitudes are never compared directly; only the prompt **sampler** mixes (across groups within a step). The builder offers a balanced round-robin interleave or a proportional shuffle, and replicates the pool so one epoch spans the run (keeping save-only-at-end intact).
- **Run status — honest.** This is the **newest** demo: the builder is in place but the dataset is **not yet built**, and it carries **no committed run at all** (a ready-to-run recipe only). So there are **no measured results** to report — unlike the fan-out's smoke baselines, mixtask has not been exercised end-to-end yet.
- **What it would measure.** Because it is generation-bound like the CPU families it mixes, the scaling story is `demo_cpu_single`'s (scale inference). Its *distinct* signal is a **per-family `eval/<family>` breakdown** rising together under one shared policy — the evidence that a single run is training a generalist rather than trading one family off against another. Note the seeds here are the **hand-authored templated `task_families.py` families**, not the fan-out's real HF datasets, so it demonstrates the multi-family training *mechanics*, not dataset-

scale coverage. Building the mixed dataset and running it to a multi-family plateau is its open work (§“Experiment plan”).

8 Open problems in production agentic RL — infrastructure and algorithms

The workflow in this report is deliberately small — one policy, an in-process harness, a per-episode sandbox. Running agentic RL at *frontier* scale surfaces a set of recurring hard problems that the open reports (2024–2026) now describe in detail. This section catalogs them in two groups — infrastructure/systems and algorithms — with the mitigations people actually ship, and notes where this project’s design already meets one and where it remains open.

8.1 Infrastructure and systems

Long-tail stragglers and partial rollouts. Rollout lengths are heavily right-skewed, so a synchronous step idles the whole cluster waiting on the last few long trajectories — and generation is more than 90 % of RL runtime [20]. Mitigations: *partial rollouts* that cap output and resume the unfinished portion next iteration (Kimi k1.5 [17]; Kimi K2 [5]); *active partial rollouts* that over-provision and recycle interrupted requests (APRIL, +22–44 % rollout throughput, now in slime [28]); *interruptible generation* that resumes decoding after a weight update (AReaL [18]); *skewness-aware routing* of long prompts (StreamRL [20]); and *dynamic sampling* that drops zero-gradient groups (DAPO [14]).

Weight-sync cost and staleness. Trainer and inference are separate backends; after each optimizer step, new weights must reach every rollout replica. Synchronously this is a serialization bubble; at 1T scale it moves hundreds of GB. Mitigations: *fast weight transfer* (Moonshot’s checkpoint-engine updates all of Kimi-K2’s ~1T parameters in ~20 s [29]); *zero-redundancy resharding* between train and generate layouts (veRL / HybridFlow [19]); and *decoupled, “one-step-off” async RL with a bounded staleness* and a staleness-aware objective (AReaL, up to ~2.77× [18]; Asynchronous RLHF [21]). *This project uses slime’s fully-async rollout (`MODE=B`), so staleness is the accepted cost — and it is exactly the signal that is instrumented-yet-unrecorded here (§“The shared skeleton”).*

Goodput vs MFU, and placement. An iteration interleaves a bandwidth-bound, long-tailed generation phase with a compute-bound training phase, so one engine's GPUs idle while the other works — which is why MFU is misleading for RL and practitioners reason in *goodput* (useful samples per GPU-second). *This project's measured MFU of 2–4 % (§demo_cpu_single) is a concrete instance of that gap.* Mitigations: *colocate-with-offload* (veRL / HybridFlow, the frontier default [19], used by Kimi K2 [5]) vs *disaggregated placement* for heterogeneous or elastic hardware (StreamRL, up to 2.66× [20]; slime supports both). *Here the 4-train + 4-inference colocation is the simple default; disaggregation is the lever once the sandbox or a second cluster enters the picture.*

Scaling execution sandboxes to tens of thousands. Every step runs untrusted code in an isolated sandbox, so the sandbox layer — not the GPU — often becomes the bottleneck: constant create/destroy, strong isolation, real startup cost. Kimi K2 runs more than 10,000 concurrent Kubernetes sandboxes for its coding / SWE RL [5]. Mitigations: *microVM isolation* (Firecracker-class, sub-second boot) and *managed remote code-exec backends* (E2B, Modal). *This is precisely AgentCore Runtime's per-session microVM — the seam this whole report builds on; the project scales it by round-robining a pool of Runtime ARNs (sticky per episode) and, on GPU, the Ray fractional farm (§demo_gpu_shared).*

Cache-aware routing. Rollout throughput lives on prefix KV-cache reuse; a round-robin balancer scatters prefix-sharing requests and collapses the hit rate — acute in agentic RL, where a group of trajectories shares one task prefix and multi-turn rollouts grow a long shared prefix. Mitigations: RadixAttention plus SGLang's cache-aware router (~1.9× throughput; hit rate ~20 %→~75 %) [27], and session-affinity routing for multi-turn (slime `consistent_hashing` [9]). *This is the reasoning behind §"Scaling the sandbox": one cache-aware SGLang router for inference, round-robin only on the sandbox side.*

Fault tolerance. One step couples training + inference + reward + thousands of live sandboxes, so the blast radius is large. Mitigations: *async checkpointing*; *worker heartbeats* (slime's `/health_generate` with restart-and-re-sync, `--use-fault-tolerance` [9]); and *sandbox recycling* with timeouts and rate limits so one bad tool call does not abort a rollout. *The project's two resilience demos (§demo_cpu_quiesce, §demo_gpu_snapshot) address the sandbox-side failure modes — idle reclamation and instance recycle.*

8.2 Algorithms

The train/inference logprob mismatch and off-policy correction. Even with identical parameters, the inference engine and the trainer assign different token probabilities (differing parallelism, precision, kernels), so "on-policy" RL is silently off-policy — biasing gradients and risking entropy collapse, and worse for long responses. Mitigations: *Truncated Importance Sampling* (clip π_learner/π_sampler, now in veRL) [22]; *fixing the precision root cause* by reverting to FP16 [23]; and *decoupled / staleness-aware PPO* [18]. *This project's TITO property (the proxy records the exact sampled `token_ids` + logprobs, no re-tokenization — §"Token-in-token-out") removes the tokenization half of the mismatch, but not the engine-vs-trainer numeric half; TIS is the missing piece, and the importance-sampling ratio is instrumented-but-unrecorded here.*

GRPO and its variants. GRPO drops PPO's critic by normalizing advantages within a group of K sampled responses [13], but has documented failure modes (entropy collapse, length bias, zero-gradient groups, MoE instability). Upgrades: *DAPO* (clip-higher ε_high≈0.28, dynamic sampling, token-level loss, overlong reward shaping) [14]; *Dr. GRPO* (removes the length and std normalization that inflate wrong-answer length) [16]; *GSPO* (a sequence-level, length-normalized importance ratio that stabilizes MoE RL and underpins Qwen3) [15]. *This project trains with GRPO; these are the drop-in improvements as runs scale up.*

Credit assignment over long horizons. An agentic episode is many steps but the verifiable reward is sparse and terminal, so trajectory-level GRPO mis-credits the actions inside a failed or successful trajectory. Mitigations: *hierarchical / turn-level grouping* (GiGPO, +9–12 % over GRPO on ALFWorld / WebShop) [26]; entropy-aware step attribution; and denser process rewards derived from outcome labels alone. *The demos use a binary end-of-episode reward with `EPISODE_MAX_TURNS=3`; long-horizon families (SWE, web) are where turn-level credit becomes necessary.*

Reward hacking and verifier trust. Any learned reward can be gamed, so frontier labs prefer rule-based verifiable rewards — DeepSeek-R1 explicitly avoids neural reward models to prevent reward hacking [2]. But verifiers are not automatically trustworthy: "Spurious Rewards" shows RLVR with *random* rewards still improved a Qwen math model, because a clipping bias amplifies pretraining behaviors rather than reward correctness — and the effect is model-dependent [24]. *The project's rewards are binary RLVR checkers run inside the sandbox (no LLM judge), which*

sidesteps neural-RM hacking; the caution is that a benchmark gain does not by itself prove the checker is doing the work.

Entropy collapse. During long RL, entropy drops early and exploration dies; performance is effectively "traded from policy entropy" [25]. Mitigations: clip-higher [14]; covariance-targeted regularization (Clip-Cov / KL-Cov) that intervenes only on the few high-covariance tokens driving entropy decay [25]; and KL control or removal.

Curriculum, dynamic sampling, and the discard ratio. All-correct or all-wrong groups give zero advantage → wasted compute. Mitigations: *dynamic sampling* to keep only groups with 0 < correct < G (DAPO [14]) and *pass-rate difficulty curricula* (AdaRFT, up to 2× less training time [30]). *This is exactly where the project's `discard_ratio` metric would come alive — it is defined (goodput depends on it) but zero in the current runs.*

8.3 Takeaways

Most of these are already partly handled by the stack this report uses: slime provides async rollout, fault-tolerant SGLang workers, and cache-aware routing; AgentCore provides the isolated, scalable sandbox. The open work for a production run concentrates in three places — (1) turning on and *recording* the off-policy signals (staleness, importance-sampling ratio, discard ratio) and adding TIS; (2) moving from GRPO to a variant (DAPO / GSPO) as scale and response length grow; and (3) turn-level credit assignment for the long-horizon families. These are algorithm choices layered on top of the infrastructure the demos already exercise.

9 Summary

- **Can:** an isolated per-session sandbox for untrusted model output (CPU by default), fully in-account (IAM/SigV4, VPC), long sessions, bring-your-own container, and **GPU on Instances**; a clean one-hook integration into slime with the harness in-process.
- **Cannot:** a microVM has no GPU; there is no cross-session GPU fractioning (and no MIG on L4); and a session cannot exceed the microVM's 8 h cap without moving to Instances (14 d).

- **How to run:** build a sandbox image → deploy a Runtime (or a GPU capacity provider) → seed a task → `invoke_agent_runtime` per tool call → the in-sandbox checker gives the reward → slime trains via a single `generate` hook. CPU↔GPU is one env var.
- **Grounding:** the concepts and lifecycle are cross-checked between the official AgentCore documentation [D1–D13, D17] and AWS's own SDK, CLI, and samples [D14–D16]; the code is this project's `agentcore_slime_demo/` (harness, sandbox, tasks), `deploy/` (Dockerfiles + `deploy_two_runtime.py`), and `demos/`.

10 References — papers & models

Agentic RL & models. [1] Agent Lightning — Harnessed Agentic RL. https://arxiv.org/abs/2508.03680 ·[2] DeepSeek-R1. https://arxiv.org/abs/2501.12948 ·[3] Qwen3 Technical Report. https://arxiv.org/abs/2505.09388 ·[4] Llama-Nemotron. https://arxiv.org/abs/2505.00949 ·[5] Kimi K2. https://arxiv.org/abs/2507.20534 ·[6] SWE-bench. https://arxiv.org/abs/2310.06770 ·[6b] SWE-bench README (arm64 "experimental"; `--task-repo` local Buildx build). https://github.com/SWE-bench/SWE-bench ·[7] KernelBench. https://huggingface.co/datasets/ScalingIntelligence/KernelBench ·[8] MBPP. https://arxiv.org/abs/2108.07732 ·[9] slime. https://github.com/THUDM/slime ·[10] AceReason-Nemotron: Advancing Math and Code Reasoning through RL. https://arxiv.org/abs/2505.16400 ·[11] LiveCodeBench. https://arxiv.org/abs/2403.07974 ·[12] NVIDIA Nemotron Post-Training Dataset v2. https://huggingface.co/datasets/nvidia/Nemotron-Post-Training-Dataset-v2

11 References — AgentCore docs (1/2)

Official AgentCore docs. [D1] What is AgentCore. https://docs.aws.amazon.com/bedrock-agentcore/latest/devguide/what-is-bedrock-agentcore.html ·[D2] Host with Runtime (isolation, compute types). https://docs.aws.amazon.com/bedrock-agentcore/latest/devguide/agents-tools-runtime.html ·[D3] Runtime how it works (idle 15 m / max 8 h). https://docs.aws.amazon.com/bedrock-agentcore/latest/devguide/runtime-how-it-works.html ·[D4] Lifecycle settings. https://docs.aws.amazon.com/bedrock-agentcore/latest/devguide/runtime-lifecycle-settings.html ·[D5] Instances (GPU). https://docs.aws.amazon.com/bedrock-agentcore/latest/devguide/runtime-instances-how-it-works.html ·[D6] Code Interpreter. https://docs.aws.amazon.com/bedrock-agentcore/latest/devguide/code-interpreter-tool.html ·[D7] Code Interpreter session. https://docs.aws.amazon.com/bedrock-agentcore/latest/devguide/code-interpreter-session-characteristics.html

12 References — AgentCore docs (2/2)

[D8] InvokeAgentRuntime. https://docs.aws.amazon.com/bedrock-agentcore/latest/APIReference/API_InvokeAgentRuntime.html ·[D9] CreateAgentRuntime. https://docs.aws.amazon.com/bedrock-agentcore-control/latest/APIReference/API_CreateAgentRuntime.html ·[D10] NetworkConfiguration. https://docs.a

ws.amazon.com/bedrock-agentcore-control/latest/APIReference/API_NetworkConfiguration.html ·[D11] VPC interface endpoints. https://docs.aws.amazon.com/bedrock-agentcore/latest/devguide/vpc-interface-endpoints.html ·[D12] StopRuntimeSession. https://docs.aws.amazon.com/bedrock-agentcore/latest/APIReference/API_StopRuntimeSession.html ·[D13] Observability. https://docs.aws.amazon.com/bedrock-agentcore/latest/devguide/observability.html ·[D17] Runtime service contract (port 8080, /invocations, /ping). https://docs.aws.amazon.com/bedrock-agentcore/latest/devguide/runtime-service-contract.html

13 References — AgentCore SDK, CLI & samples

AWS code, cross-checked against the docs. [D14] AgentCore samples (`awslabs/agentcore-samples`). https://github.com/awslabs/agentcore-samples ·[D15] AgentCore Python SDK (`bedrock-agentcore`; `BedrockAgentCoreApp` / `@app.entrypoint`). https://github.com/aws/bedrock-agentcore-sdk-python ·[D16] AgentCore CLI (`@aws/agentcore`, recommended) https://www.npmjs.com/package/@aws/agentcore ·legacy starter toolkit https://github.com/aws/bedrock-agentcore-starter-toolkit

14 References — production agentic RL (open problems)

RL algorithms & systems. [13] DeepSeekMath (GRPO). https://arxiv.org/abs/2402.03300 ·[14] DAPO. https://arxiv.org/abs/2503.14476 ·[15] GSPO (Group Sequence Policy Optimization). https://arxiv.org/abs/2507.18071 ·[16] Understanding R1-Zero-Like Training (Dr. GRPO). https://arxiv.org/abs/2503.20783 ·[17] Kimi k1.5. https://arxiv.org/abs/2501.12599 ·[18] AReaL (asynchronous RL system). https://arxiv.org/abs/2505.24298 ·[19] HybridFlow (veRL). https://arxiv.org/abs/2409.19256 ·[20] StreamRL (disaggregated stream generation). https://arxiv.org/abs/2504.15930 ·[21] Asynchronous RLHF. https://arxiv.org/abs/2410.18252 ·[22] Truncated Importance Sampling / train–inference mismatch (Yao et al.). https://fengyao.notion.site/off-policy-rl ·[23] Defeating the Training–Inference Mismatch via FP16. https://arxiv.org/abs/2510.26788 ·[24] Spurious Rewards. https://arxiv.org/abs/2506.10947 ·[25] The Entropy Mechanism of RL. https://arxiv.org/abs/2505.22617 ·[26] GiGPO (hierarchical group advantages). https://arxiv.org/abs/2505.10978 ·[27] SGLang / RadixAttention. https://arxiv.org/abs/2312.07104 — cache-aware router: https://lmsys.org/blog/2024-12-04-sglang-v0-4/ ·[28] APRIL (active partial rollouts). https://arxiv.org/abs/2509.18521 ·[29] Moonshot checkpoint-engine. https://github.com/MoonshotAI/checkpoint-engine ·[30] AdaRFT (difficulty curriculum). https://arxiv.org/abs/2504.05520